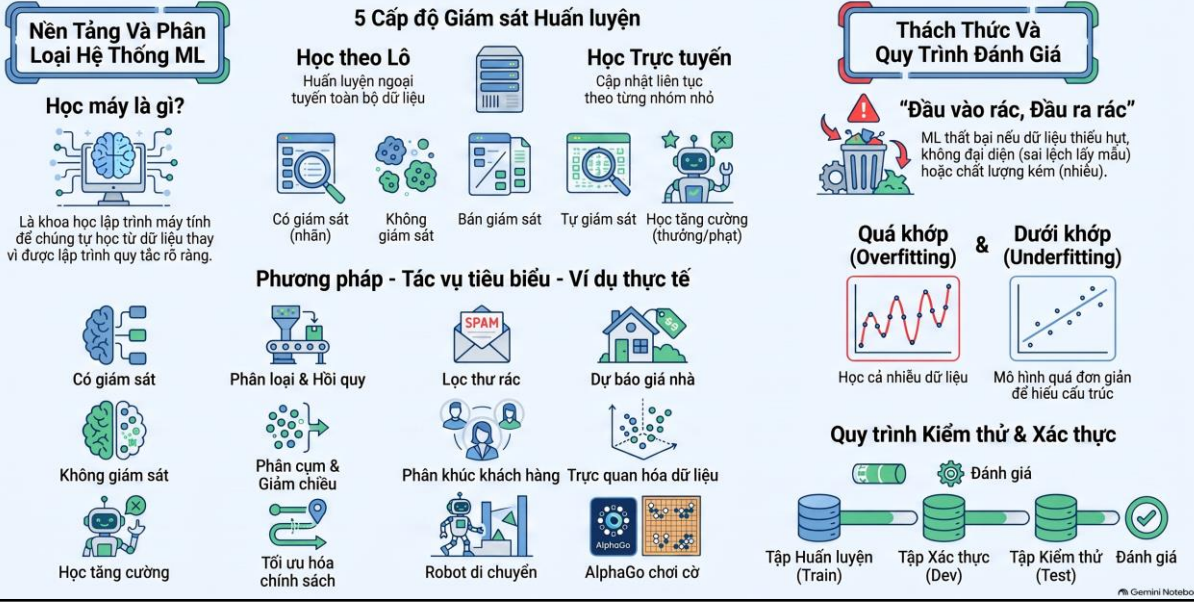
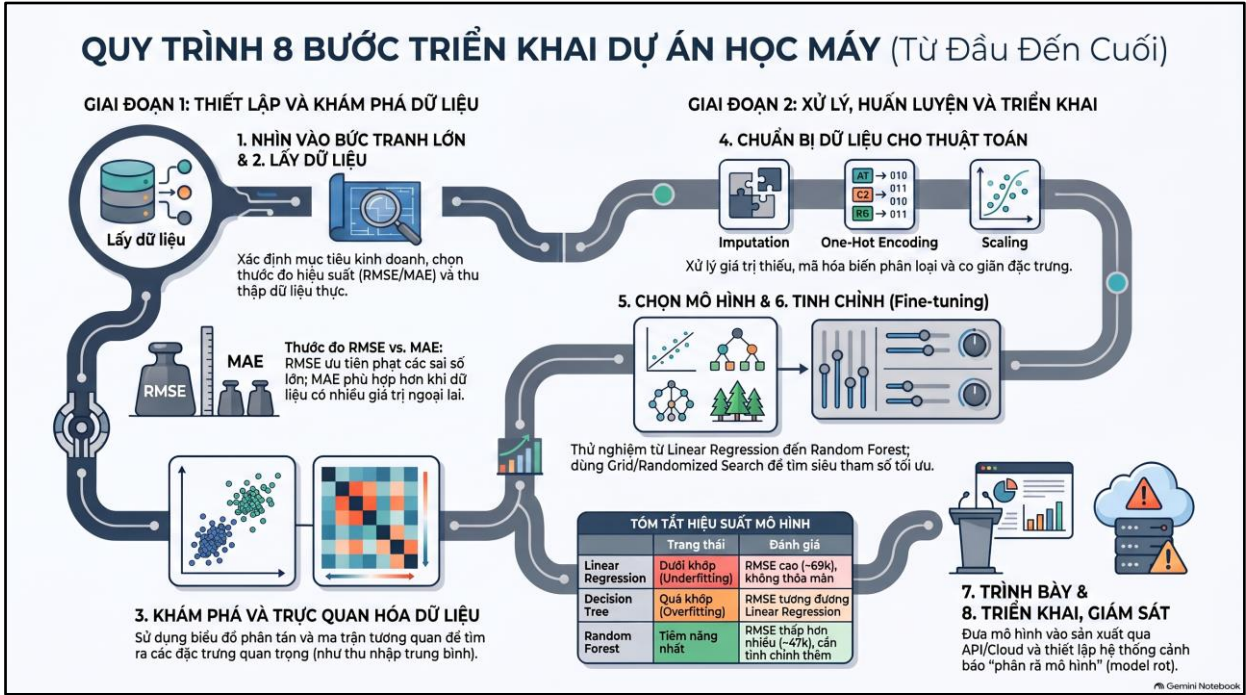


Lộ Trình Khám Phá Học Máy: Từ Lý Thuyết Đến Thực Tiễn





TỔNG QUAN VỀ PHÂN LOẠI TRONG MACHINE LEARNING (Chương 3)

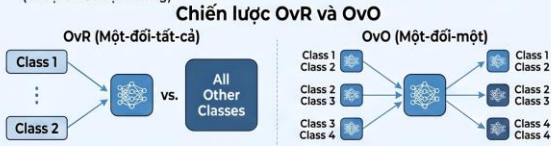
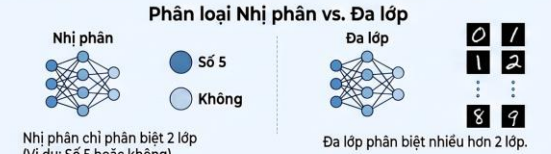
Tóm tắt các khái niệm cốt lõi về phân loại dữ liệu, từ bộ dữ liệu mẫu đến các phương pháp đánh giá mô hình.

CÁC LOẠI HÌNH PHÂN LOẠI & DỮ LIỆU MNIST

MNIST - "Hello World" của ML

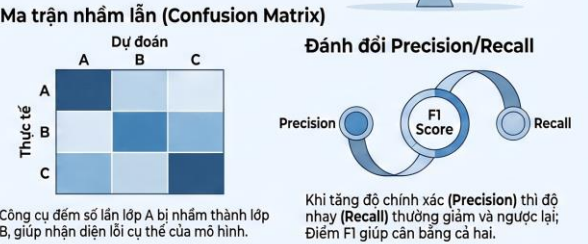


Bộ dữ liệu gồm 70.000 hình ảnh chữ số viết tay (28x28 pixel) dùng để huấn luyện mô hình.



Cách kết hợp nhiều bộ phân loại nhị phân để giải quyết bài toán đa lớp.

ĐÁNH GIÁ HIỆU SUẤT MÔ HÌNH





DỰ ÁN HỌC MÁY TỪ ĐẦU ĐẾN CUỐI

Chương 3 - Phân loại (Classification)



TS. Trần Thành Thắng

Đại học Đông Á

August 11, 2026

© Gemini Notebook

Nội dung Chương trình

1

1. Giới thiệu Phân loại, MNIST & Đánh giá Hiệu suất cơ bản

2

2. Sự đánh đổi Hiệu suất & Phân loại Đa lớp

3

3. Phân tích lỗi, Đa nhãn & Đa đầu ra

© Gemini Notebook

Mục tiêu bài học

- ✓ Nắm vững khái niệm Phân loại (Classification) trong Học máy.
- ✓ Hiểu rõ và thực hành với bộ dữ liệu MNIST ("Hello World" của học máy).
- ✓ Đánh giá mô hình bằng các thước đo hiệu suất (Ma trận nhầm lẫn, Precision, Recall, F1-score, ROC AUC).
- ✓ Phân biệt và thực hiện Phân loại Đa lớp (Multiclass), Đa nhãn (Multilabel) và Đa đầu ra (Multioutput).
- ✓ Kỹ năng phân tích lỗi để cải thiện hiệu suất mô hình.



© Gemini Notebook



1. Giới thiệu Phân loại, MNIST & Đánh giá Hiệu suất cơ bản

© Gemini Notebook



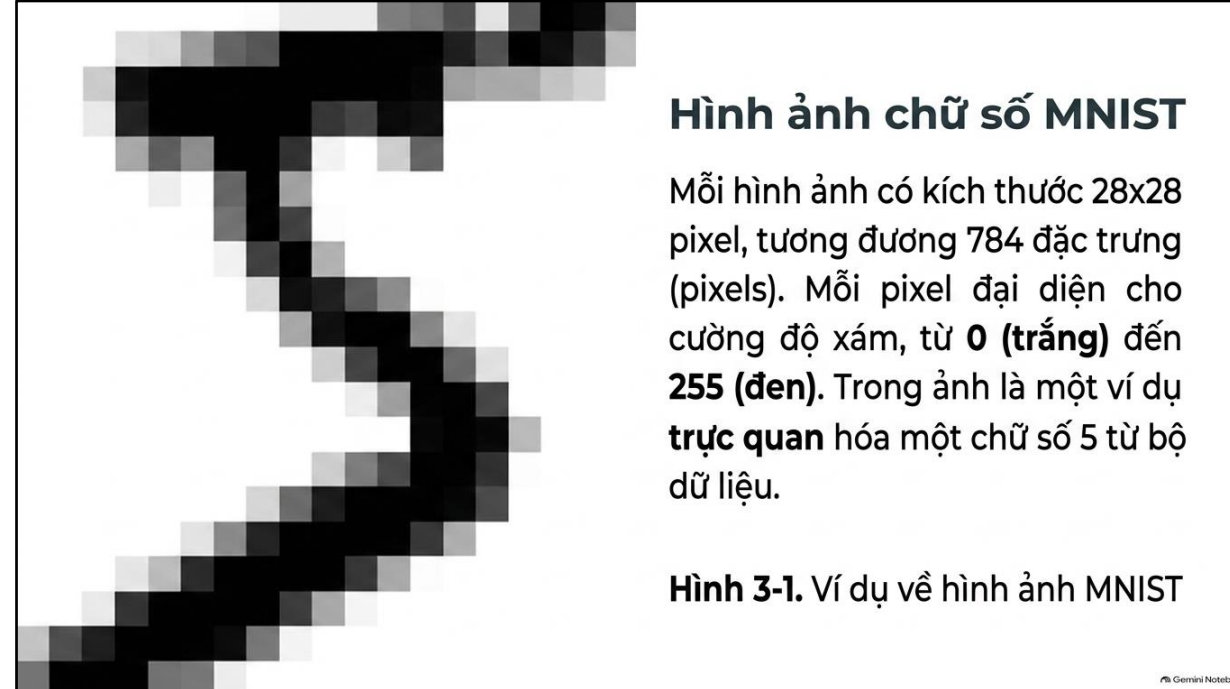
Giới thiệu Bộ dữ liệu MNIST

Các tác vụ học có giám sát phổ biến nhất là Hồi quy (Regression) và Phân loại (Classification). Chúng ta sẽ sử dụng bộ dữ liệu MNIST, gồm 70.000 hình ảnh nhỏ của các chữ số viết tay bởi học sinh và nhân viên Cục Thống kê Hoa Kỳ.

MNIST được nghiên cứu nhiều đến mức nó được gọi là "Hello World" của học máy.

Mục tiêu: Phân loại (nhận dạng) chữ số trong từng bức ảnh.

© Gemini Notebook



5 0 4 1 9 2 1 3 1 4
 3 5 3 6 1 7 2 8 6 9
 4 0 9 1 1 2 4 3 2 7
 3 8 6 9 0 5 6 0 7 6
 1 8 7 9 3 9 8 5 9 3
 3 0 7 4 9 8 0 9 4 1
 4 4 6 0 4 5 6 1 0 0
 1 7 1 6 3 0 2 1 1 7
 9 0 2 6 7 8 3 9 0 4
 6 7 4 6 8 0 7 8 3 1

Sự đa dạng của chữ số MNIST

Các chữ số được viết với nhiều phong cách, nét thanh, nét đậm khác nhau. Để minh họa độ phức tạp của bài toán phân loại, hình bên cạnh liệt kê một số chữ số mẫu từ tập dữ liệu.



Hình 3-2. Các chữ số từ bộ dữ liệu MNIST

© Gemini Notebook

Phân chia tập huấn luyện và tập kiểm thử



Nguyên tắc vàng: Luôn tạo một tập kiểm thử (**Test set**) và tách biệt nó trước khi kiểm tra kỹ lưỡng dữ liệu. May mắn là tập dữ liệu MNIST tải từ OpenML đã được chia sẵn:

- Tập huấn luyện (Train set): 60.000 hình ảnh đầu tiên.
- Tập kiểm thử (Test set): 10.000 hình ảnh cuối cùng.

Tập huấn luyện đã được xáo trộn (shuffled), đảm bảo các thuật toán học (đặc biệt là các thuật toán học tuần tự) không bị nhận nhiều trường hợp giống nhau liên tiếp.

© Gemini Notebook



Huấn luyện bộ phân loại nhị phân (Binary Classification)

© Gemini Notebook



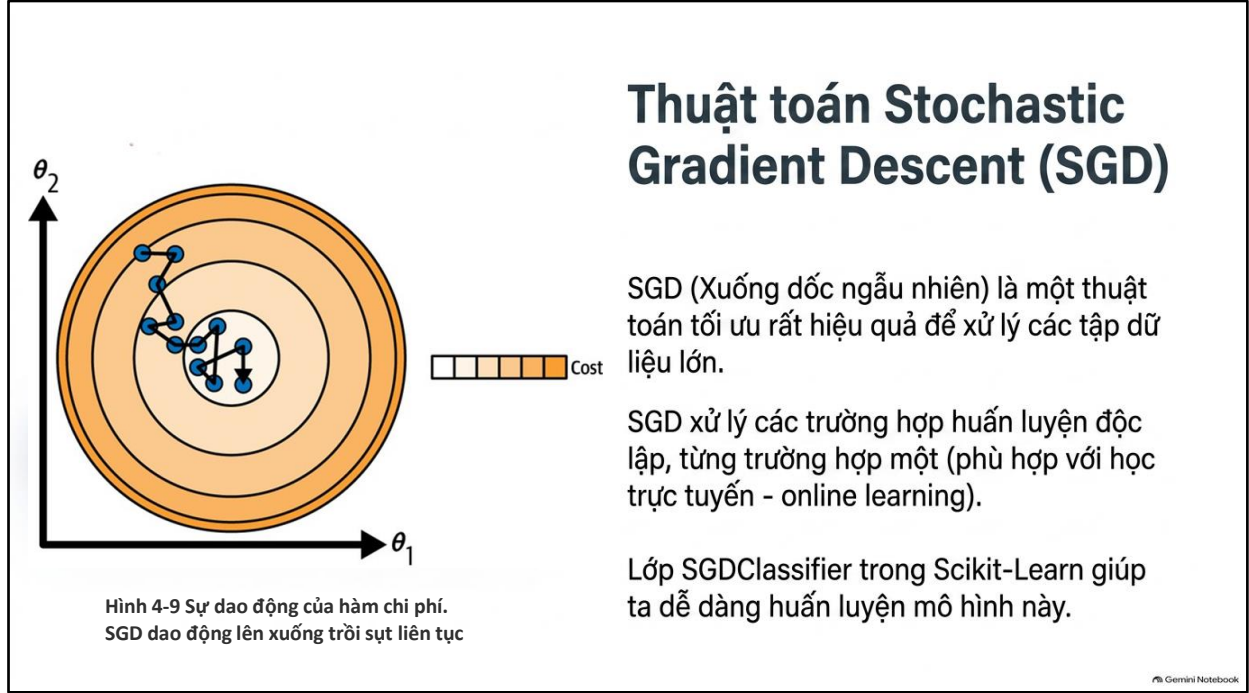
Bài toán: Bộ phát hiện chữ số 5

Hãy đơn giản hóa bài toán: Chúng quy chỉ cần phát hiện xem một chữ số có phải là "5" hay không.

Đây là ví dụ về Phân loại Nhị phân (Binary Classification): Phân biệt giữa 2 lớp (5 và không-phải-5).

Mục tiêu (Label) mới: True (nếu là số 5), False (nếu là số khác).

© Gemini Notebook



Stochastic Gradient Descent (SGD - Hạ độ dốc ngẫu nhiên) là một thuật toán tối ưu hóa lặp lại mạnh mẽ, được sử dụng rộng rãi trong cả Học máy cổ điển và Học sâu (Deep Learning) để điều chỉnh các tham số của mô hình nhằm giảm thiểu hàm chi phí [cite: 25, 91, 207]. Dưới đây là giải thích chi tiết, toàn diện về bản chất, cách thức hoạt động, ưu nhược điểm cũng như so sánh của thuật toán này dựa trên các tài liệu học tập của bạn:

1. Ý tưởng cốt lõi và Cách thức hoạt động

Trong thuật toán **Batch Gradient Descent (BGD)** truyền thống, ở mỗi bước lặp, mô hình bắt buộc phải sử dụng *toàn bộ* tập dữ liệu huấn luyện để tính toán vector gradient, điều này khiến quá trình cực kỳ chậm và tốn bộ nhớ RAM khi dữ liệu trở nên khổng lồ [cite: 104, 217].

Để giải quyết triệt để điểm nghẽn này, **SGD hoạt động theo nguyên lý đối lập** [cite: 104, 217]:

Cách thực hiện: Tại mỗi bước huấn luyện, SGD lựa chọn ngẫu nhiên **chỉ một mẫu dữ liệu duy nhất** (single instance) từ tập huấn luyện để tính toán gradient và thực hiện cập nhật các tham số (θ) [cite: 104, 107, 217, 220].

Công thức bước hạ gradient:
$$\theta^{(\text{bước tiếp theo})} = \theta - \eta \nabla_{\theta} J(\theta)$$
 Trong đó η là tốc độ học (learning rate), và gradient chỉ được tính trên một mẫu ngẫu nhiên duy nhất thay vì chia cho tổng số mẫu m [cite: 7, 100, 107, 213, 220].

Hiện tượng dao động và Lịch trình học tập (Learning Schedule)

Do thuật toán chỉ dựa vào một mẫu ngẫu nhiên duy nhất ở mỗi bước, hướng đi của SGD không hề thẳng tắp hay mượt mà hướng tới cực tiểu toàn cục [cite: 105, 218]. Hàm chi phí sẽ **dao động lên xuống liên tục** (nhảy loạn xạ) và chỉ có xu hướng giảm khi tính trung bình [cite: 105, 218].

Khi tiến sát đến cực tiểu, thuật toán sẽ tiếp tục nhảy múa xung quanh điểm tối ưu mà **không bao giờ dừng yên hay thực sự hội tụ** [cite: 105, 181, 218]. Khi dừng lại, giá trị tham số thu được thường rất tốt nhưng không phải là tối ưu tuyệt đối [cite: 105, 218].

Để khắc phục hiện tượng này, người ta áp dụng một cơ chế gọi là **Lịch trình học tập (learning schedule)** [cite: 106, 181, 219]:

Ban đầu, tốc độ học được đặt ở mức **lớn** để giúp thuật toán tiến bộ nhanh chóng và dễ dàng nhảy qua (thoát khỏi) các cực tiểu cục bộ nông [cite: 106, 219].

Càng về sau, tốc độ học sẽ được **giảm dần** theo thời gian (giống như quá trình mô phỏng ủ kim loại), giúp các bước cập nhật nhỏ lại và cho phép mô hình dần ổn định, hội tụ tại cực tiểu toàn cục [cite: 106, 183, 219].

2. Ưu điểm nổi bật của SGD

Tốc độ huấn luyện cực nhanh: Do chỉ tính toán trên một mẫu duy nhất tại một thời điểm, SGD có vòng lặp huấn luyện nhanh nhất và tiếp cận vùng lân cận của giải pháp tối ưu sớm nhất [cite: 15, 183].

Khả năng mở rộng quy mô khổng lồ: Nhờ yêu cầu tài nguyên tối thiểu (chỉ cần một mẫu duy nhất nằm trong bộ nhớ tại mỗi lần lặp), SGD hỗ trợ cực tốt cho việc huấn luyện các tập dữ liệu khổng lồ không vừa RAM thông qua giải thuật **học ngoài bộ nhớ (out-of-core learning)** [cite: 104, 185, 217].

Phù hợp cho học trực tuyến (Online Learning): SGD có thể cập nhật mô hình một cách độc lập và tuần tự theo thời gian thực ngay khi có dữ liệu mới xuất hiện [cite: 89, 194, 205].

Thoát khỏi cực tiểu cục bộ: Sự dao động ngẫu nhiên mạnh mẽ hoạt động như một "cú hích", giúp thuật toán dễ dàng vượt qua các cao nguyên phẳng hoặc nhảy ra khỏi các cực tiểu cục bộ không tối ưu trong các hàm chi phí phức tạp phi tuyến [cite: 106, 219].

3. Nhược điểm và Thách thức

Nghiệm không tối ưu tuyệt đối: Nếu không thiết lập một lịch học tốt, mô hình sẽ liên tục dao động xung quanh điểm tối ưu mà không bao giờ thực sự hội tụ ổn định [cite: 105, 218].

Nhạy cảm cực kỳ với việc co giãn đặc trưng (Feature Scaling): Giống như các thuật toán Gradient Descent khác, nếu các đặc trưng có thang đo quá lệch nhau, hàm chi phí sẽ kéo dẹt thành hình "chiếc bát thuôn dài" [cite: 179, 211]. Khi đó, SGD sẽ đi vòng vèo, dao động dữ dội qua lại hai bên thung lũng phẳng và mất cực kỳ nhiều thời gian để mò xuống đáy [cite: 179, 211]. Việc chuẩn hóa dữ liệu (ví dụ bằng StandardScaler) trước khi huấn luyện là bước bắt buộc để đạt hiệu năng tốt [cite: 179, 187].



Các thước đo hiệu suất

© Gemini Notebook

Đo độ chính xác bằng kiểm định chéo

Giống như hồi quy, ta có thể dùng kiểm định chéo (Cross-Validation) để đánh giá mô hình phân loại.

Kết quả: SGDClassifier đạt **độ chính xác (Accuracy) trên 95% cho mọi fold!**
Nghe có vẻ tuyệt vời, nhưng hãy xem xét cẩn thận hơn.



© Gemini Notebook



Sự đánh lừa của Độ chính xác (Accuracy)

Nếu ta tạo một "bộ phân loại ngốc nghếch" (Dummy Classifier) luôn đoán "không-phải-5", thì độ chính xác của nó cũng trên 90%!

Tại sao? Vì chỉ có 10% chữ số trong tập dữ liệu là số 5.

Kết luận: Độ chính xác (Accuracy) không phải là thước đo tốt cho các tập dữ liệu bị lệch (skewed datasets).

© Gemini Notebook

Ma trận nhầm lẫn (Confusion Matrix)

Ma trận nhầm lẫn (CM) là một cách tốt hơn nhiều để đánh giá hiệu suất.

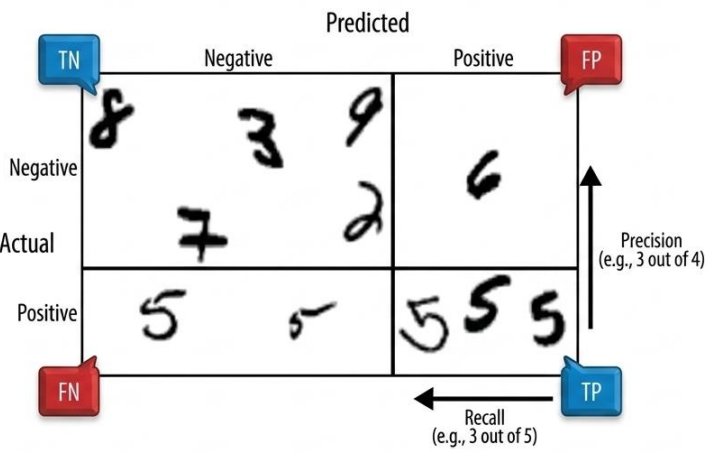
Ý tưởng chung: Đếm số lần các trường hợp của lớp A bị phân loại nhầm thành lớp B.

Mỗi hàng là một lớp thực tế, mỗi cột là một lớp được dự đoán.



© Gemini Notebook

Ma trận nhầm lẫn (Confusion Matrix)



- True Negative (TN):**
Âm tính thật.
- False Positive (FP):**
Dương tính giả (Lỗi loại I).
- False Negative (FN):**
Âm tính giả (Lỗi loại II).
- True Positive (TP):**
Dương tính thật.

Hình 3-3. Ma trận nhầm lẫn minh họa

Precision vs. Recall: Hai góc nhìn của dự đoán



Độ chính xác (Precision)

Độ chính xác của lớp Dương tính. Trong số các trường hợp mô hình dự đoán là Tích cực, có bao nhiêu phần trăm là đúng?

$$\text{Precision} = \frac{TP}{TP + FP}$$



Độ nhạy (Recall)

Tỷ lệ dương tính đúng. Trong số các trường hợp Tích cực thực sự, mô hình đã phát hiện được bao nhiêu phần trăm?

$$\text{Recall} = \frac{TP}{TP + FN}$$

© Gemini Notebook

Điểm F1 (F1 Score): Sự kết hợp hoàn hảo



Điểm F1 là trung bình điều hòa (Harmonic Mean) của Precision và Recall.

⚠️ Điểm F1 chỉ cao khi CẢ Precision và Recall đều cao. Nó ưu tiên các bộ phân loại có hai chỉ số này tương tự nhau.

$$F1 = \frac{2}{(1/Precision) + (1/Recall)} = 2 \times \frac{(Precision \times Recall)}{Precision + Recall}$$

Cơ chế hoạt động: Ngưỡng quyết định (Decision Threshold)

Làm sao mô hình quyết định một mẫu là Tích cực hay Tiêu cực?

Tiêu cực



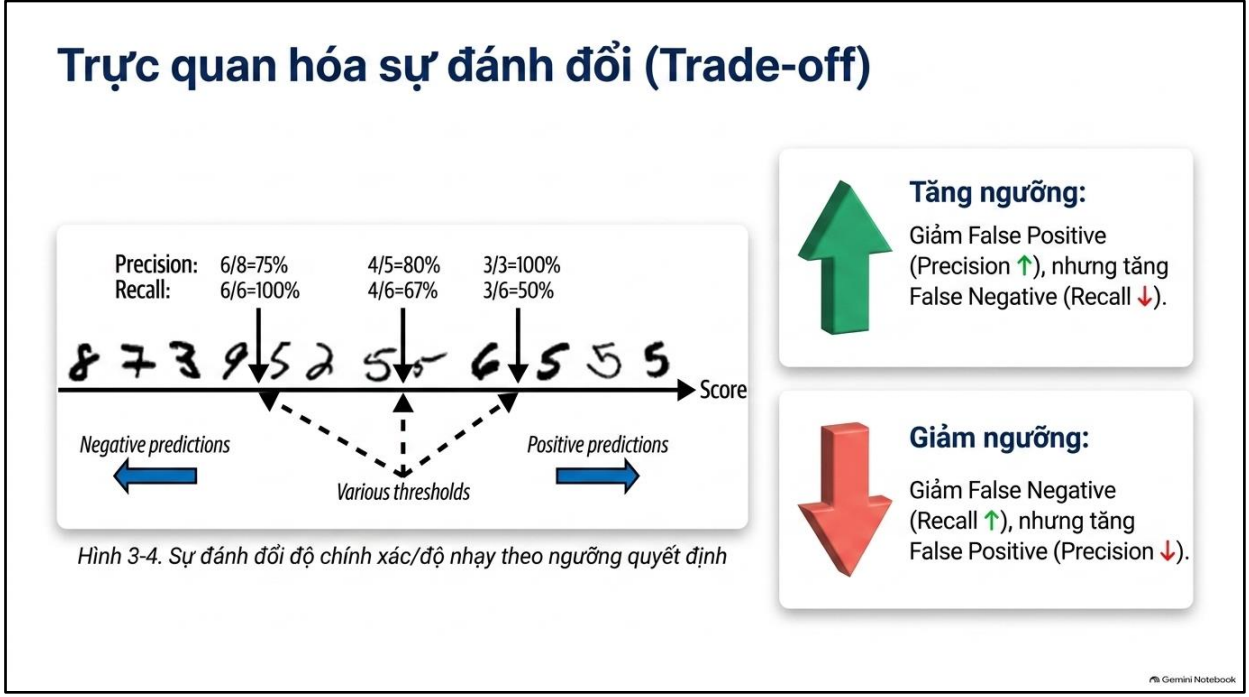
Tích cực

Mô hình tính toán một điểm số quyết định (decision score).



Nếu điểm số > Ngưỡng -> Tích cực
Nếu điểm số < Ngưỡng -> Tiêu cực

© Gemini Notebook



Thực chất, các con số **8, 7, 3, 9 | 5, 2, 5 | 5, 6 | 5, 5, 5** mà bạn nhìn thấy **không phải là các vạch chia giá trị của ngưỡng quyết định hay thang điểm (score)** [cite: 58, 177, 220]. Đó chính là **hình ảnh của các chữ số viết tay thực tế** từ bộ dữ liệu MNIST đang được xếp hàng từ trái sang phải [cite: 58, 177, 220].

1. Bản chất của trục hoành "Score" là gì?
- Khi bạn đưa các hình ảnh chữ số viết tay vào mô hình **SGDClassifier**, mô hình sẽ tính toán cho mỗi hình ảnh một **Điểm quyết định (Decision Score)** thô [cite: 58, 174, 220]: Điểm số này thể hiện **độ tin cậy** của mô hình: Điểm càng cao thì mô hình càng tin chắc hình ảnh đó là số 5 [cite: 58, 175, 220].
- Trong **Hình 3-4**, tác giả đã **sắp xếp 12 bức ảnh chữ số viết tay theo thứ tự điểm số tăng dần từ trái sang phải** [cite: 58, 177, 220].
- Phía bên trái (điểm số thấp): Là những chữ số mà mô hình chắc chắn *không phải là số 5* (như các số **8, 7, 3, 9**) [cite: 177, 178].
 - Ở giữa: Là những số dễ gây nhiễu, viết hơi giống số 5 (như số **2** viết tay — mà bạn nhìn nhầm thành số **7** — và số **6** viết hơi ngoằn ngoèo) [cite: 58, 177, 178, 220].
 - Phía bên phải (điểm số cao): Là các số **5** viết rất rõ ràng, mô hình cực kỳ tự tin [cite: 58, 220].

2. Sự đánh đổi Độ chính xác (Precision) và Độ nhạy (Recall) hoạt động thế nào?
- Để quyết định nhãn cuối cùng là **True (là số 5)** hay **False (không phải số 5)**, mô hình cần so sánh điểm số với một **Ngưỡng quyết định (Decision Threshold)** [cite: 58, 175, 220]. Những hình ảnh nằm **bên phải ngưỡng** được đoán là **Positive (số 5)** [cite: 58, 177, 220].
- Sơ đồ minh họa **3 mũi tên thẳng đứng** đại diện cho 3 kịch bản đặt ngưỡng khác nhau để bạn thấy rõ sự đánh đổi [cite: 58, 177, 220]:
- Kịch bản 1: Đặt ngưỡng thấp (Mũi tên bên trái — nằm giữa số 9 và số 5) [cite: 178]
- Vùng dự đoán là số 5 (bên phải ngưỡng):** Chứa tất cả 6 chữ số 5 thực tế, cộng thêm số 2 và số 6 bị đoán sai (tổng cộng 8 ảnh) [cite: 178].
- Độ chính xác (Precision):** $\frac{6 \text{ (số 5 đúng)}}{8 \text{ (tổng số đoán là 5)}} = \mathbf{75\%}$ [cite: 178]

Độ nhạy (Recall): Vì ngưỡng thấp nên mô hình không bỏ sót bất kỳ số 5 thực tế nào: $\frac{6}{6} = \mathbf{100\%}$ [cite: 178]

Kịch bản 2: Đặt ngưỡng trung tâm (Mũi tên ở giữa — nằm giữa số 2 và số 5 tiếp theo) [cite: 58, 178]

Vùng dự đoán là số 5 (bên phải ngưỡng): Chỉ còn lại 4 chữ số 5 thực tế và 1 chữ số 6 bị đoán sai (tổng cộng 5 ảnh) [cite: 58, 178, 220].

Độ chính xác (Precision): $\frac{4}{5} \text{ (số 5 đúng)} \div 5 \text{ (tổng số đoán là 5)} = \mathbf{80\%}$ (tăng từ 75% lên 80%) [cite: 58, 178, 220]

Độ nhạy (Recall): Do ngưỡng tăng lên, mô hình đã bỏ sót mất 2 chữ số 5 nằm ở bên trái ngưỡng: $\frac{4}{6} \approx \mathbf{67\%}$ (sụt giảm từ 100% xuống 67%) [cite: 58, 178, 220]

Kịch bản 3: Đặt ngưỡng cao (Mũi tên bên phải — nằm giữa số 6 và số 5 tiếp theo) [cite: 58, 178]

Vùng dự đoán là số 5 (bên phải ngưỡng): Chỉ còn đúng 3 chữ số 5 viết cực kỳ rõ ràng (tổng cộng 3 ảnh) [cite: 58, 178, 220].

Độ chính xác (Precision): Không mắc bất kỳ lỗi dương tính giả nào: $\frac{3}{3} = \mathbf{100\%}$ (độ tin cậy tuyệt đối) [cite: 58, 178, 220]

Độ nhạy (Recall): Đổi lại, mô hình đã bỏ sót một nửa số lượng số 5 thực tế trong tập dữ liệu: $\frac{3}{6} = \mathbf{50\%}$ [cite: 58, 178, 220]

Tóm lại: Khi bạn **tăng ngưỡng** (dịch chuyển ranh giới quyết định sang bên phải), điều kiện để được coi là "số 5" trở nên khắt khe hơn. Điều này giúp mô hình **ít đoán sai hơn (Precision tăng)** nhưng lại **bỏ sót nhiều mục tiêu thực tế hơn (Recall giảm)** [cite: 58, 179, 220]. Đây chính là **sự đánh đổi bất biến** trong mọi bài toán phân loại [cite: 58, 176, 220]!

1. Ý nghĩa của "Recall" (Độ nhạy / Độ bao phủ)

Recall trả lời cho câu hỏi: *“Trong số tất cả các mục tiêu thực tế đang tồn tại trong dữ liệu, mô hình đã tìm ra và bao phủ được bao nhiêu phần trăm?”* [cite: 83, 211, 260].

Công thức: $\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$ [cite: 84, 212, 261]

TP (True Positives - Dương tính đúng): Những mục tiêu thực tế được mô hình tìm thấy chính xác [cite: 81, 207, 258].

FN (False Negatives - Âm tính giả): Những mục tiêu thực tế nhưng bị mô hình đoán sai thành âm tính (tức là **bị bỏ sót**) [cite: 81, 207, 258].

2. Tại sao tăng ngưỡng quyết định lại gây "bỏ sót mục tiêu" (Recall giảm)?

Khi bạn **tăng ngưỡng quyết định** (dịch vạch chia sang bên phải trong Hình 3-4), mô hình chỉ chấp nhận gán nhãn "Dương tính" (ví dụ: là số 5) cho các bức ảnh có điểm số cực kỳ cao và rõ ràng [cite: 87, 221, 264].

Hậu quả của sự khắt khe này là:

Những chữ số 5 thực tế nhưng viết hơi cầu thả, mờ hoặc lệch nét sẽ **không đạt đủ điểm số** để vượt qua ngưỡng mới [cite: 89, 217, 265].

Chúng bị mô hình phân loại nhầm thành "Không phải số 5" (gây ra lỗi **Âm tính giả - FN**) [cite: 81, 207, 258].

Về mặt thực tế, điều này có nghĩa là mô hình **bỏ sót (miss)** các chữ số 5 này [cite: 89, 212, 265]. Do tử số TP giảm và mẫu số FN tăng, chỉ số **Recall tự động bị kéo sụt giảm** [cite: 87, 221, 264].

3. Minh họa trực quan từ Hình 3-4 [cite: 88, 219, 265]

Giả sử tập dữ liệu của bạn có tổng cộng **6 chữ số 5 thực tế** nằm rải rác trên trục điểm số [cite: 87, 220, 264]:

Ở ngưỡng thấp (Mũi tên bên trái): Mô hình dễ dãi, quét sạch mọi thứ bên phải. Nó bắt được cả 6 số 5.

Bỏ sót: 0 số.

Recall = $6/6 = \mathbf{100\%}$ [cite: 178, 220].

Ở ngưỡng trung tâm (Mũi tên ở giữa): Mô hình khắt khe hơn một chút. Nó chỉ nhận diện được 4 số 5 rõ ràng. Còn 2 số 5 viết hơi xấu ở bên trái vạch chia bị bỏ qua [cite: 58, 220].

Bỏ sót: 2 mục tiêu thực tế [cite: 58, 220].

Recall giảm xuống còn $4/6 \approx \mathbf{67\%}$ [cite: 58, 220].

Ở ngưỡng cao (Mũi tên bên phải): Mô hình cực kỳ khắt khe, chỉ chấp nhận 3 số 5 viết cực đẹp [cite: 58, 220].

Bỏ sót: 3 mục tiêu thực tế (bỏ sót mất một nửa mục tiêu cần tìm) [cite: 58, 220].

Recall tụt mạnh xuống còn $3/6 = \mathbf{50\%}$ [cite: 58, 220].

Chiến lược phân tích biểu đồ



Mục tiêu là chọn được một **Threshold** phù hợp nhất với bài toán thực tế.

Chiến lược 1:

Vẽ Precision và Recall theo từng giá trị ngưỡng (Threshold).
(Giống Hình 3-4)

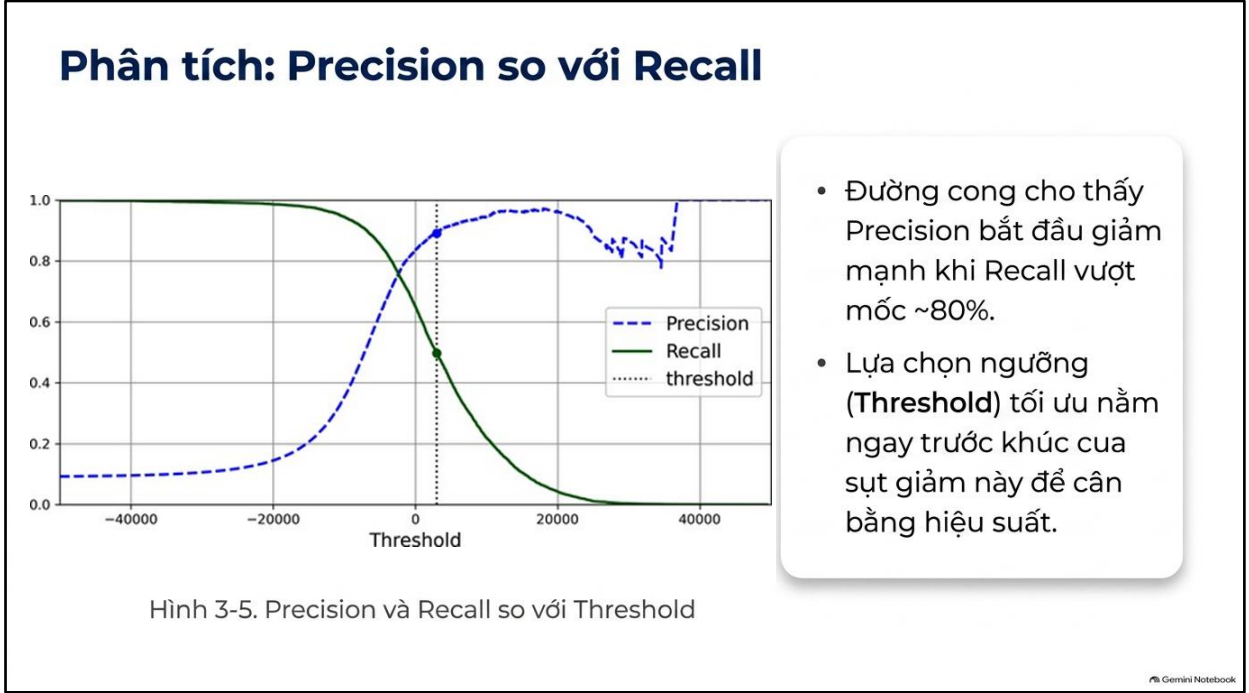
Chiến lược 2:

Vẽ trực tiếp Precision theo Recall để xem đường tiệm cận.



Điểm vàng: Nên chọn điểm ngưỡng ngay trước khi Precision bị sụt giảm quá mạnh (ví dụ: ở Recall khoảng **60%-80%**).

© Gemini Notebook



Sự quan sát của bạn rất tinh tế! Khi nhìn vào **Hình 3-5 (Đồ thị Độ chính xác và Độ nhạy theo ngưỡng quyết định)**, bạn sẽ thấy một hiện tượng thú vị: **Đường màu xanh dương (Precision) thì nhấp nhô, gồ ghề (bumpy)**, trong khi **đường màu xanh lá (Recall) lại đi xuống một cách cực kỳ mượt mà** [cite: 165]. Thực chất, sự "ngoằn ngoèo" này chỉ xảy ra chủ yếu ở đường **Độ chính xác (Precision)**, còn đường **Độ nhạy (Recall)** luôn đi xuống một cách trơn tru [cite: 165]. Dưới đây là lý do toán học và trực quan đằng sau hiện tượng này:

1. Tại sao đường Độ nhạy (Recall) luôn mượt mà?

Độ nhạy (Recall) được tính theo công thức [cite: 158]:
$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

Mẫu số (TP + FN) chính là **tổng số lượng mục tiêu thực tế** có trong tập dữ liệu (tổng số chữ số 5 thực tế) [cite: 158, 165]. Đây là một **hằng số cố định**, hoàn toàn không thay đổi bất kể bạn đặt ngưỡng quyết định ở đâu [cite: 165].

Khi bạn **tăng ngưỡng quyết định** (đi từ trái sang phải trên trục hoành), mô hình ngày càng khắt khe hơn [cite: 161, 165]. Do đó, số lượng chữ số được mô hình đoán là số 5 chỉ có thể giảm đi hoặc giữ nguyên, khiến số lượng Dương tính đúng (**TP**) **chỉ có thể giảm xuống liên tục** [cite: 165].

Vì tử số (TP) liên tục giảm trong khi mẫu số không đổi, **Recall bắt buộc phải đi xuống một chiều một cách mượt mà và không bao giờ có chuyện nhô lên** khi bạn tăng ngưỡng [cite: 165].

2. Tại sao đường Độ chính xác (Precision) lại nhấp nhô, ngoằn ngoèo?

Độ chính xác (Precision) được tính theo công thức [cite: 156]:
$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

Khác với Recall, khi bạn tăng ngưỡng quyết định, **cả tử số (TP) và mẫu số (TP + FP) đều thay đổi** [cite: 165].

Nguyên nhân gốc rễ của sự ngoằn ngoèo: Khi bạn tăng ngưỡng lên một chút, bạn có thể **vô tình loại bỏ một mẫu Đúng (True Positive - TP) trước khi loại bỏ được mẫu Sai (False Positive - FP)** [cite: 165]. Lúc này, tử số bị giảm đi trong khi mẫu số giảm không đáng kể, làm cho tỷ lệ Precision bị **sụt giảm tạm thời tại điểm đó** (tạo ra vết răng cưa đi xuống) trước khi tiếp tục xu hướng tăng chung

[cite: 165].

3. Ví dụ thực tế trực quan từ Hình 3-4 (Chuỗi chữ số)

Để hiểu vì sao Precision bị sụt giảm khi tăng ngưỡng, hãy nhìn vào chuỗi chữ số được xếp hàng ở giữa **Hình 3-4** [cite: 58, 162]:

$$\begin{matrix} \text{\textbf{\textit{Vạch \, , ngưỡng \, , ở \, đây}}} \\ \mathbf{5} \quad \mathbf{2} \quad \mathbf{5} \quad \mathbf{6} \quad \mathbf{5} \quad \mathbf{5} \quad \mathbf{5} \end{matrix}$$

Tại vạch ngưỡng ban đầu (mũi tên ở giữa) [cite: 163]:

Bên phải vạch ngưỡng có 5 chữ số: trong đó có **4 số 5 đúng** (TP = 4) và **1 số 6 sai** (FP = 1) [cite: 163].

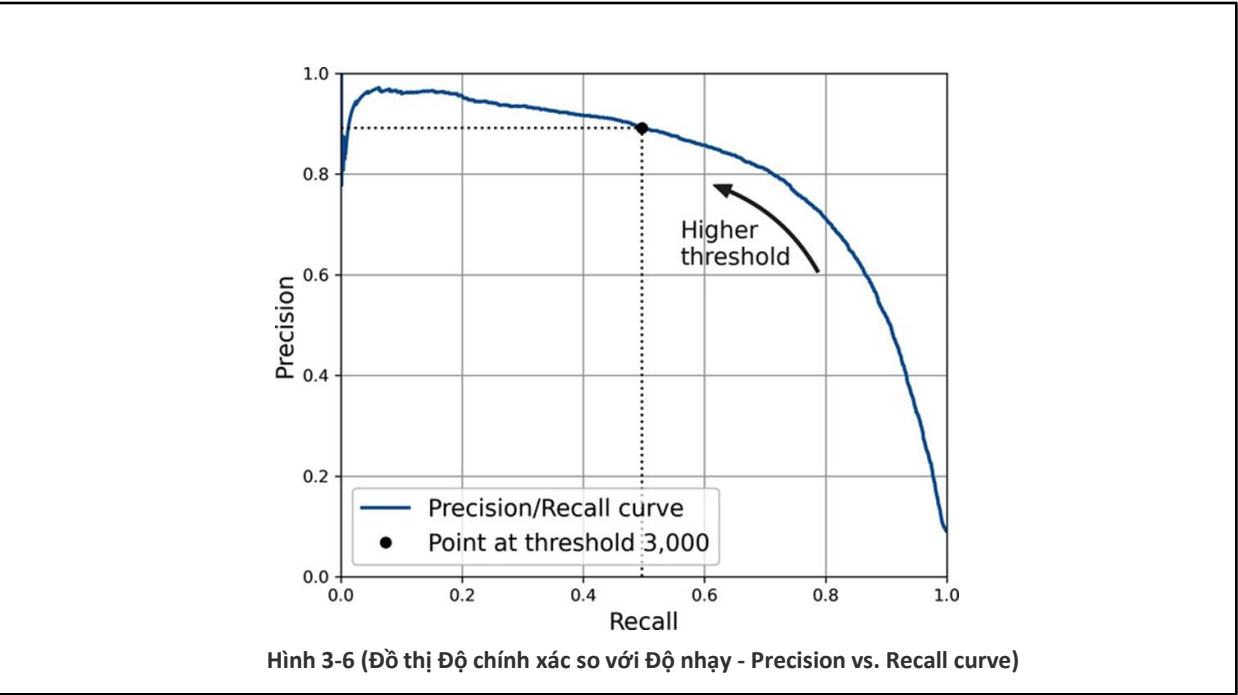
$$\text{Precision} = \frac{4}{4 + 1} = \mathbf{80\%}$$
 [cite: 163].

Bây giờ, bạn tăng ngưỡng quyết định lên một chút (dịch vạch sang bên phải, vượt qua chữ số 5 đầu tiên ở bên phải vạch cũ nhưng *chưa vượt qua* số 6):

Bên phải vạch ngưỡng mới lúc này chỉ còn lại 4 chữ số: bao gồm **3 số 5 đúng** (TP = 3) và vẫn còn nguyên **1 số 6 sai** (FP = 1) [cite: 163].

$$\text{Precision} = \frac{3}{3 + 1} = \mathbf{75\%}$$
 [cite: 163].

Kết quả: Khi bạn tăng ngưỡng quyết định, **Precision đã bị tụt từ 80% xuống 75%** [cite: 163, 165]! Sự sụt giảm này chính là điểm ngoằn ngoèo đi xuống trên đồ thị [cite: 165]. Phải cho đến khi bạn tiếp tục tăng ngưỡng vượt qua hẳn số 6 sai kia (lúc này chỉ còn các số 5 đúng, Precision đạt 100%), đường cong mới vọt lên lại [cite: 163, 165].



Hình 3-6 (Đồ thị Độ chính xác so với Độ nhạy - Precision vs. Recall curve) trong tài liệu của bạn là một trong những biểu đồ trực quan quan trọng nhất để đánh giá hiệu năng của một bộ phân loại nhị phân (như bộ phát hiện số 5) [cite: 51, 137]. Nếu như Hình 3-5 biểu diễn Precision và Recall riêng biệt theo sự thay đổi của Ngưỡng quyết định (trục hoành là Ngưỡng) [cite: 51, 137], thì **Hình 3-6 đưa chúng về đối chiếu trực tiếp với nhau trên cùng một đồ thị** (không còn trục Ngưỡng trực tiếp nữa) để bạn thấy rõ sự đánh đổi giữa hai chỉ số này [cite: 51, 137]. Dưới đây là giải thích chi tiết và trực quan về cấu trúc, ý nghĩa và cách đọc biểu đồ này:

1. Ý nghĩa các thành phần trên đồ thị Hình 3-6

Trục tung (Precision - Độ chính xác của lớp dương tính): Thể hiện khả năng mô hình dự đoán chính xác [cite: 130]. Trục này chạy từ 0.0 (0%) đến 1.0 (100%) [cite: 5].

Trục hoành (Recall - Độ nhạy / Tỷ lệ bao phủ mục tiêu): Thể hiện khả năng mô hình không bỏ sót mục tiêu thực tế [cite: 130]. Trục này cũng chạy từ 0.0 đến 1.0 [cite: 5].

Đường cong màu xanh (Precision/Recall curve): Đường này được vẽ bằng cách chạy thử mô hình qua tất cả các ngưỡng quyết định có thể có từ thấp đến cao [cite: 53, 139]. Với mỗi ngưỡng, mô hình sẽ cho ra một cặp giá trị (Recall, Precision) tương ứng và tập hợp các cặp này tạo nên đường cong [cite: 53, 139].

Chấm đen nổi bật (Point at threshold 3,000): Đây là một ví dụ thực tế khi bạn đặt ngưỡng quyết định của mô hình SGDClassifier bằng **3.000** [cite: 5].

Dóng chấm đen này sang trục tung (Precision): Bạn thu được giá trị khoảng **90%** [cite: 51, 137].

Dóng chấm đen này xuống trục hoành (Recall): Bạn thu được giá trị khoảng **50%** [cite: 51, 137].

Ý nghĩa: Với ngưỡng 3.000, cứ 10 hình ảnh mô hình báo là số 5 thì có 9 hình đúng (Precision = 90%), nhưng mô hình lại bỏ sót một nửa số lượng số 5 thực tế có trong tập dữ liệu (Recall = 50%) [cite: 58, 109].

Mũi tên cong "Higher threshold" (Ngưỡng cao hơn): Mũi tên này hướng lên trên và sang trái [cite: 5]. Nó chỉ ra rằng **khi bạn tăng ngưỡng quyết định**, điểm làm việc của mô hình sẽ trượt dọc theo đường cong về phía góc trên bên trái: **Precision sẽ tăng lên nhưng Recall buộc phải giảm xuống** [cite: 58, 109].

2. Bản chất hình dạng đường cong và Cách chọn "Điểm tối ưu"

Nhìn vào đường cong, ta thấy một đặc điểm hình học rất rõ ràng:

Vùng bình nguyên phẳng (bên trái): Khi Recall tăng từ 0.0 lên đến khoảng 0.6 (60%), đường cong nằm rất ngang và cao [cite: 137]. Điều này nghĩa là mô hình có thể tăng Recall (tìm thêm nhiều số 5 hơn) mà không làm ảnh hưởng nhiều đến độ chính xác (Precision vẫn duy trì cực kỳ cao, gần 90%) [cite: 137].

Điểm rơi tự do (vùng khuỷu tay - elbow): Bắt đầu từ khoảng **80% độ nhạy (Recall)** trở đi, đường cong **đột ngột lao dốc một cách thẳng đứng và thảm hại** [cite: 137]. Điều này phản ánh rằng nếu bạn cố ép mô hình phải truy quét để bắt cho bằng hết mọi số 5 (Recall tiến sát về 100%), mô hình bắt đầu đoán mò vô tội vạ, dẫn đến việc báo động giả tăng vọt và Precision tụt dốc không phanh [cite: 137].

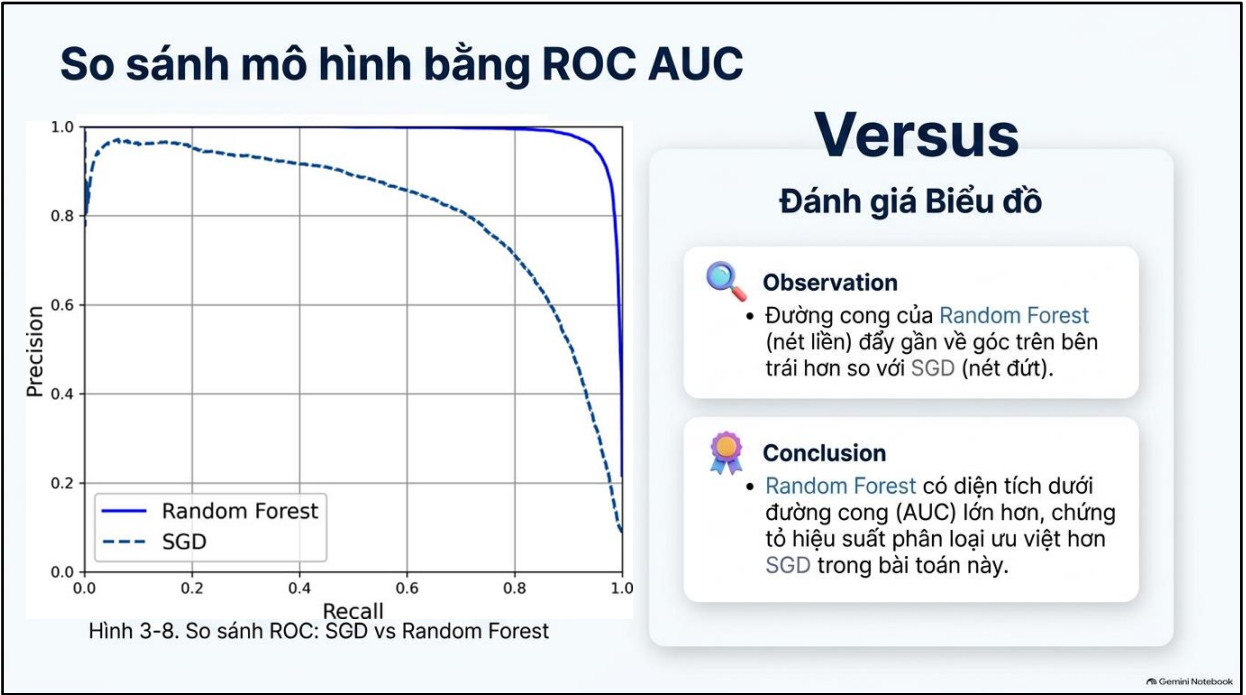
Quy tắc chọn ngưỡng thực tế: Người ta thường muốn chọn một sự đánh đổi (điểm làm việc) **ngay trước khi xảy ra cú sụt giảm mạnh đó** — ví dụ, ở khoảng **60% độ nhạy (Recall)** [cite: 137]. Tại điểm này, mô hình vừa bắt được một lượng mục tiêu tương đối tốt (60%), vừa giữ được độ chính xác rất cao (~90%) [cite: 137].

3. Khi nào bạn nên sử dụng đường cong PR (Hình 3-6) thay vì đường cong ROC (Hình 3-7)?

Tài liệu học tập của bạn nhấn mạnh một mẹo cực kỳ quan trọng khi làm dự án thực tế [cite: 52, 138]:

Bạn nên **ưu tiên dùng đường cong Precision/Recall (PR)** của Hình 3-6 khi **lớp dương tính (mục tiêu cần tìm) cực kỳ hiếm** hoặc khi bạn quan tâm nhiều đến lỗi Dương tính giả (FP) hơn là Âm tính giả (FN) [cite: 52, 56, 138, 142].

Ví dụ: Trong bài toán phát hiện số 5 từ MNIST, số 5 chỉ chiếm 10% dữ liệu (bị lệch lớp nghiêm trọng) [cite: 49]. Nếu dùng đường cong ROC, đồ thị trông sẽ rất "đẹp" và dễ đánh lừa người nhìn (như hình RandomForestClassifier đạt ROC AUC 99.8% ở Hình 3-8) [cite: 56, 142]. Nhưng khi nhìn vào đường cong PR, bạn sẽ thấy rõ mô hình thực tế vẫn còn nhiều lỗi báo động giả khi cố tăng độ bao phủ [cite: 56, 142].



3. Công cụ so sánh: Diện tích dưới đường cong (AUC - Area Under Curve)

Để biết mô hình nào thực sự "thông minh" hơn mà không cần tranh cãi về việc chọn ngưỡng, người ta đo toán học phần **diện tích nằm dưới đường cong ROC (ROC AUC)** [cite: 78, 184, 198, 212]:

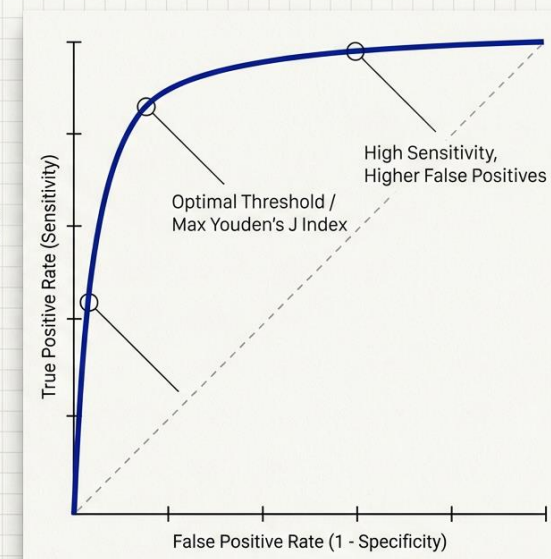
Bộ phân loại hoàn hảo (ôm sát góc trên bên trái): **AUC = 1.0** [cite: 78, 184, 198, 212].

Bộ phân loại hoàn toàn ngẫu nhiên: **AUC = 0.5** [cite: 78, 184, 198, 212].

Mô hình `SGDClassifier` của bạn đạt điểm AUC là **0.9605 (96.05%)** [cite: 79, 184, 213]. Đây là một điểm số rất tốt [cite: 79, 184, 213], nhưng khi đem so sánh với mô hình tổ hợp phức tạp hơn như **`RandomForestClassifier` (đạt AUC tới 0.9983 - Hình 3-8)**, bạn sẽ thấy rõ sự chênh lệch đẳng cấp giữa hai thuật toán [cite: 81, 187, 216].

Giải phần Đường cong ROC (Receiver Operating Characteristic)

Công cụ tối ưu để đánh giá và lựa chọn mô hình phân loại nhị phân.



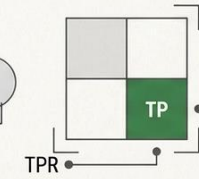
Gemini Notebook

Nền tảng cốt lõi: Hai tỷ lệ kiến tạo ROC

TPR (Độ nhạy / Recall)

Số ca được mô hình **BẮT ĐÚNG** (True Positives)

TỔNG số ca Dương tính thực tế

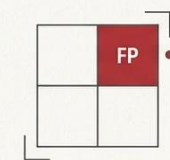


FPR (Cảnh báo nhầm / Fall-out)

Số ca bị **BÁO ĐỘNG NHẦM** (False Positives)

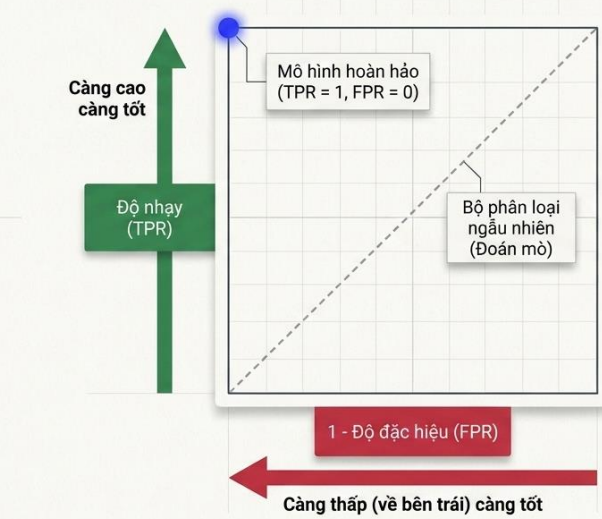
TỔNG số ca Âm tính thực tế

$FPR = 1 - \text{Specificity}$
(Độ đặc hiệu)

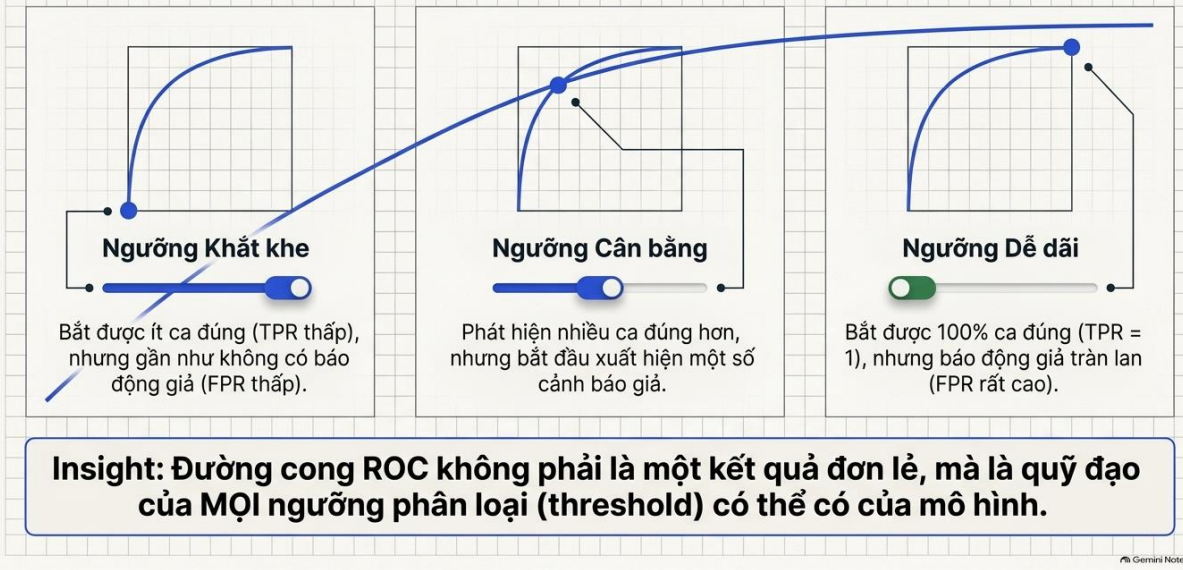


Khung tọa độ: Lắp ghép không gian ROC

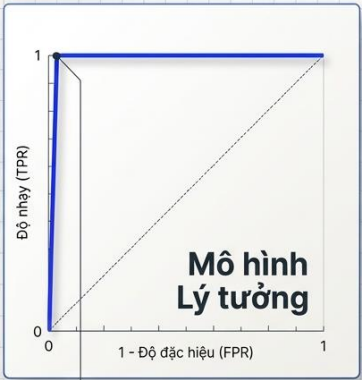
Đường cong ROC là một đồ thị 2 chiều phản ánh trực quan sự đối đầu giữa tỷ lệ phát hiện đúng (Trục tung) và tỷ lệ báo động giả (Trục hoành) của một mô hình.



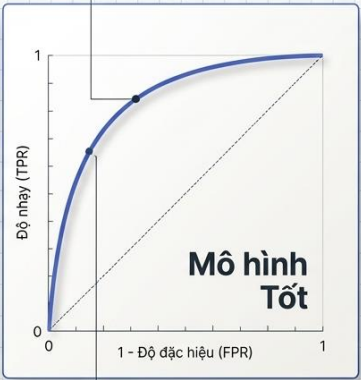
Bản chất sự vận động: Quỹ đạo của Ngưỡng quyết định



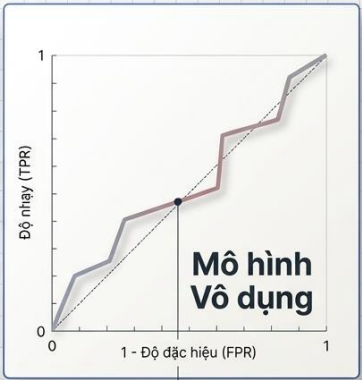
Ma trận Chẩn đoán: Đọc vị hình dáng Đường cong



Ôm sát góc trên cùng bên trái.
Phân tách hoàn hảo 100% dữ liệu.

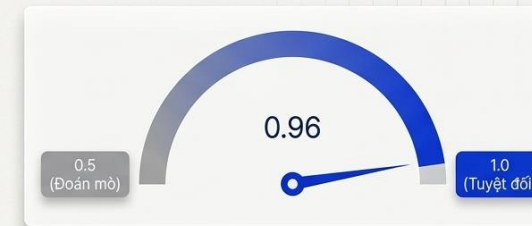
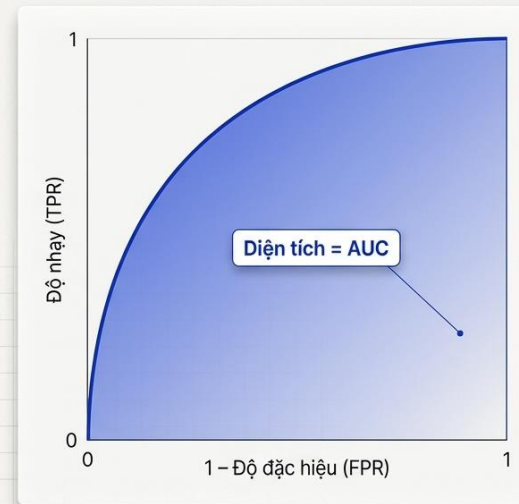


Cung tròn vươn xa khỏi đường
chéo tâm. Khả năng phân biệt cao.



Bám sát đường chéo $y=x$.
Hiệu suất tương đương việc tung
đồng xu đoán mò.

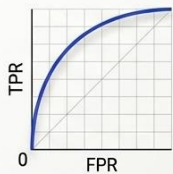
Đúc kết thành một con số: Diện tích AUC (Area Under the Curve)



- **Định nghĩa:** AUC là một thước đo chuẩn hóa duy nhất đại diện cho hiệu suất tổng thể của mô hình.
- **Ý nghĩa:** Nó đo lường xác suất mô hình sẽ xếp hạng một mẫu dương tính ngẫu nhiên cao hơn một mẫu âm tính ngẫu nhiên.
- **Ưu điểm:** Khách quan và hoàn toàn độc lập với việc lựa chọn ngưỡng quyết định.

Bức tranh toàn cảnh: Khi nào ROC bị đánh lừa?

Đường cong ROC

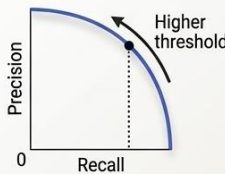


Trục tọa độ: TPR vs. FPR.

Môi trường lý tưởng: Sử dụng khi tập dữ liệu CÂN BẰNG (số ca Âm tính và Dương tính xấp xỉ nhau).

Đặc điểm: Cung cấp góc nhìn tổng quan, đối xử bình đẳng với cả hai lớp.

Đường cong PR (Precision/Recall)



Trục tọa độ: Precision vs. Recall.

Môi trường lý tưởng: Sử dụng khi tập dữ liệu MẤT CÂN BẰNG NGHIÊM TRỌNG (ví dụ: phát hiện gian lận, ung thư hiếm nghèo).

Cảnh báo: Khi số ca dương tính rất hiếm, ROC có thể cho ra một kết quả mí mắt giả tạo (ảo tưởng sức mạnh). PR Curve sẽ phơi bày sự thật.

Bí kíp bỏ túi: Tổng hợp Đường cong ROC



1. Tọa độ cốt lõi

Trục tung là Độ nhạy (TPR - Bắt đúng bao nhiêu).
Trục hoành là Cảnh báo giả (FPR - Báo nhầm bao nhiêu). Góc trên bên trái là thiên đường.



3. Thước đo vàng (AUC)

Lượng hóa đồ thị thành một con số duy nhất bằng cách tính diện tích dưới đường cong. Thang điểm từ 0.5 (Tệ nhất) đến 1.0 (Tuyệt đối).



2. Bản chất hình học

Không phải một điểm đơn lẻ. ROC vẽ ra một đường cong biểu diễn mọi kịch bản Ngưỡng quyết định (Threshold) có thể áp dụng cho mô hình.

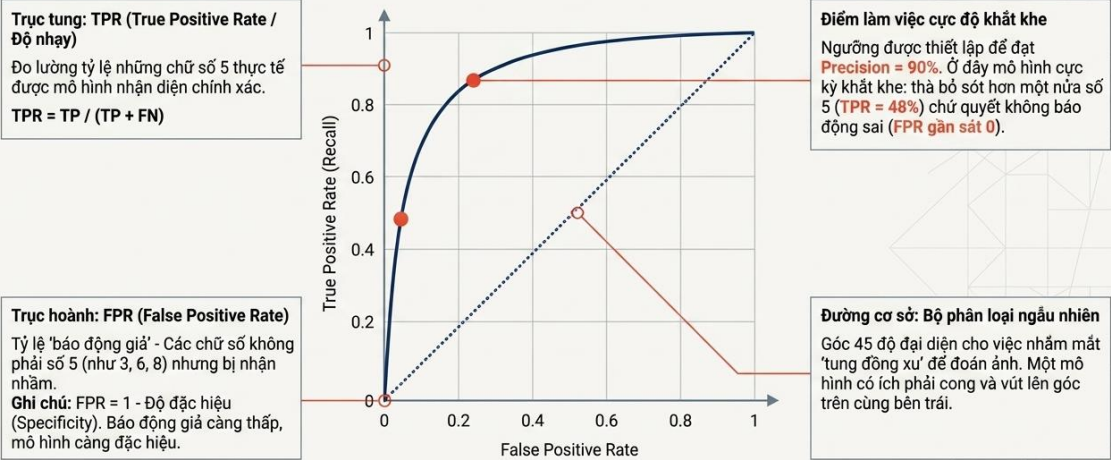


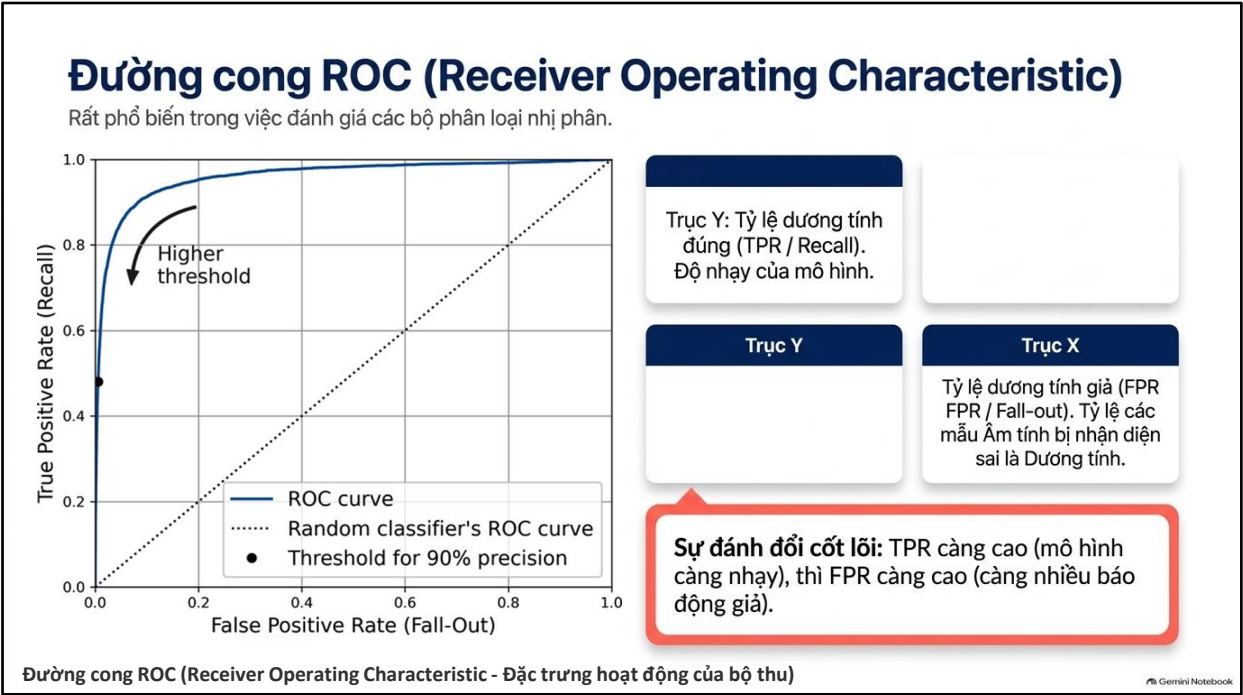
4. Quy tắc ngoại lệ

Nếu bạn đang tìm “kim đáy biển” (dữ liệu dương tính cực kì hiếm), hãy cắt ROC đi và sử dụng Đường cong PR (Precision/Recall Curve).

BẢN ĐỒ GIẢI PHẪU ĐƯỜNG CONG ROC

Khác với đường cong PR chỉ tập trung vào lớp Dương tính, ROC đối chiếu trực tiếp khả năng bắt trúng mục tiêu với tỷ lệ báo động giả.





Đường cong ROC (Receiver Operating Characteristic - Đặc trưng hoạt động của bộ thu) trong **Hình 3-7** là một công cụ đồ họa kinh điển và cực kỳ phổ biến để đánh giá hiệu năng của các bộ phân loại nhị phân (như mô hình phát hiện số 5) [cite: 75, 180, 209]. Nếu như đường cong PR (Hình 3-6) tập trung vào lớp Dương tính (Precision vs. Recall) [cite: 75, 209], thì đường cong ROC đối chiếu trực tiếp khả năng **bắt trúng mục tiêu** với tỷ lệ **báo động giả** của mô hình [cite: 75, 78, 180, 183, 209, 212]. Dưới đây là giải thích chi tiết từng thành phần và ý nghĩa vật lý của Hình 3-7:

1. Ý nghĩa các trục tọa độ

Trục tung - TPR (True Positive Rate / Tỷ lệ dương tính đúng) [cite: 76, 182, 209]: Đây chính là **Recall (Độ nhạy)** mà chúng ta đã thảo luận [cite: 76, 181, 209]. Nó đo lường tỷ lệ những chữ số 5 thực tế được mô hình nhận diện chính xác [cite: 76, 181, 209].
$$\text{TPR} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$
 [cite: 181]

Trục hoành - FPR (False Positive Rate / Tỷ lệ dương tính giả) [cite: 76, 181, 209]: Đo lường tỷ lệ các chữ số không phải số 5 (như số 3, 6, 8...) nhưng bị mô hình **nhận nhầm (báo động giả)** thành số 5 [cite: 76, 181, 209].
$$\text{FPR} = \frac{\text{FP}}{\text{TN} + \text{FP}} = 1 - \text{TNR (Specificity - Độ đặc hiệu)}$$
 [cite: 76, 182, 209] *Vì vậy, đường cong ROC còn được gọi là đồ thị biểu diễn mối quan hệ giữa Độ nhạy và (1 - Độ đặc hiệu) [cite: 76, 182, 209].*

2. Giải mã các thành phần trên đồ thị Hình 3-7

Đường cong màu xanh dương (ROC curve) [cite: 8, 183]: Đây là tập hợp tất cả các điểm làm việc của mô hình SGDClassifier khi ta chạy thử qua mọi ngưỡng quyết định có thể [cite: 76, 183, 210, 211].

Bản chất của sự đánh đổi: Đường cong này đi lên và sang phải [cite: 8, 183]. **TPR càng cao (mô hình càng nhạy) thì FPR càng cao (mô hình càng tạo ra nhiều báo động giả)** [cite: 78, 183, 198, 212].

Tại sao? Khi bạn hạ thấp ngưỡng để cố gắng không bỏ sót bất kỳ số 5 nào (TPR tiến dần về 1.0), mô hình sẽ trở nên cực kỳ "dễ dãi" và bắt đầu đoán bừa, dẫn đến việc nhận nhầm các chữ số khác

thành số 5 tăng vọt (FPR tăng nhanh) [cite: 78, 183, 197, 212].

Đường chấm chéo màu đen (Random classifier's ROC curve) [cite: 8, 183]: Đường thẳng góc 45 độ này đại diện cho một **bộ phân loại hoàn toàn ngẫu nhiên** (giống như việc bạn chỉ tung đồng xu để đoán xem ảnh có phải là số 5 hay không) [cite: 78, 183, 212].

Một mô hình học máy thực sự có ích phải nằm **càng xa đường chấm chéo này càng tốt** và hướng sát về phía **góc trên cùng bên trái** (nơi TPR gần 100% và FPR gần 0%) [cite: 78, 183, 212].

Chấm đen nổi bật (Threshold for 90% precision) [cite: 8, 183]: Điểm này tương ứng với ngưỡng quyết định được chọn để đạt độ chính xác Precision = 90% ở phần trước [cite: 77, 183, 210, 211].

Tại chấm đen này, **TPR (Recall) chỉ đạt 48%**, nhưng đổi lại **FPR (báo động giả) cực kỳ thấp (gần sát trục tung 0.0)** [cite: 78, 183, 212]. Điều này có nghĩa là mô hình rất khắt khe: thà bỏ sót một nửa số 5 chứ quyết không báo động sai [cite: 78, 183, 212].

Mũi tên cong "Higher threshold" (Ngưỡng cao hơn) [cite: 8, 183]: Hướng trượt dọc theo đường cong đi xuống phía dưới bên trái [cite: 8, 183]. Nó chỉ ra rằng **khi bạn tăng ngưỡng quyết định**, mô hình khắt khe hơn nên cả tỷ lệ nhận diện đúng (TPR) và tỷ lệ báo động giả (FPR) đều sẽ giảm đồng thời [cite: 78, 183, 197, 212].

3. Công cụ so sánh: Diện tích dưới đường cong (AUC - Area Under Curve)

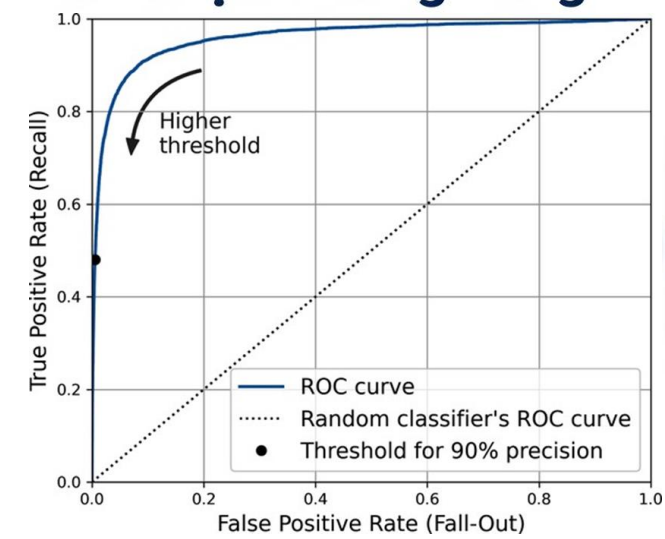
Để biết mô hình nào thực sự "thông minh" hơn mà không cần tranh cãi về việc chọn ngưỡng, người ta đo toán học phần **diện tích nằm dưới đường cong ROC (ROC AUC)** [cite: 78, 184, 198, 212]:

Bộ phân loại hoàn hảo (ôm sát góc trên bên trái): **AUC = 1.0** [cite: 78, 184, 198, 212].

Bộ phân loại hoàn toàn ngẫu nhiên: **AUC = 0.5** [cite: 78, 184, 198, 212].

Mô hình SGDClassifier của bạn đạt điểm AUC là **0.9605 (96.05%)** [cite: 79, 184, 213]. Đây là một điểm số rất tốt [cite: 79, 184, 213], nhưng khi đem so sánh với mô hình tổ hợp phức tạp hơn như **RandomForestClassifier (đạt AUC tới 0.9983 - Hình 3-8)**, bạn sẽ thấy rõ sự chênh lệch đẳng cấp giữa hai thuật toán [cite: 81, 187, 216].

Minh họa Đường cong ROC



Hình dạng đường cong mô tả trực quan mức độ hiệu quả của mô hình so với một bộ phân loại ngẫu nhiên.

© Gemini Notebook



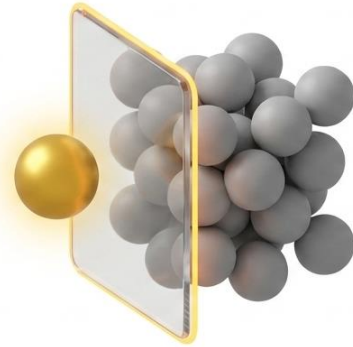
Mở rộng bài toán: Phân loại đa lớp (Multiclass)

Khái niệm vượt ra ngoài phân loại nhị phân (chỉ có Tích cực/Tiêu cực).

Thách thức: Phân biệt giữa nhiều hơn hai lớp cùng một lúc (Ví dụ: Nhận diện chữ số viết tay từ 0 đến 9).

© Gemini Notebook

Chiến lược Một-đối-phần-còn-lại (OvR / OvA)



OvR (One-vs-Rest): Huấn luyện nhiều bộ phân loại nhị phân độc lập.



Để nhận diện số từ 0-9:
- Huấn luyện 10 mô hình.
- Mô hình "0": Số 0 vs. Các số còn lại.



Khi phân loại, **lấy điểm quyết định từ cả 10 mô hình và chọn lớp có điểm cao nhất.**

Chiến lược Một-đối-một (OvO)



OvO (One-vs-One): Huấn luyện một bộ phân loại nhị phân cho MỖI CẶP nhãn.
Ví dụ: 0 vs 1, 0 vs 2, 1 vs 2...

Tổng số mô hình cần huấn luyện:

$$N \times (N - 1) / 2$$

(Ví dụ: 10 chữ số = 45 bộ phân loại!)

Phù hợp cho các thuật toán khó mở rộng với lượng dữ liệu lớn (như Support Vector Machine - SVC).

© Gemini Notebook

Tự động hóa thông minh với Scikit-Learn



Scikit-Learn tự động phát hiện khi ta dùng mô hình nhị phân cho dữ liệu đa lớp.

Thư viện tự động chọn chiến lược OvO hoặc OvR tùy thuộc vào đặc tính của thuật toán.

Ví dụ: Truyền toàn bộ các nhãn `y_train` (từ 0-9) vào mô hình Support Vector Machine (SVC), hệ thống sẽ tự động huấn luyện bằng phương pháp OvO tối ưu.

© Gemini Notebook



Phân loại đa lớp với thuật toán gốc

Một số mô hình sinh ra đã hỗ trợ Phân loại đa lớp. Trong trường hợp này, Scikit-Learn không cần phải tạo ra các thuật toán nhị phân giả lập (OvO/OvR).

RandomForestClassifier

SGDClassifier

GaussianNB

© Gemini Notebook

Điểm số quyết định cho dự đoán đa lớp



INPUT

Gọi hàm
`decision_function()`
cho mô hình đa lớp.



PROCESSING

Mô hình trả về 1 mảng
điểm tương ứng với
mọi lớp có thể.



OUTPUT

Điểm cao nhất nằm ở
vị trí (index) nào thì mô
hình sẽ dự đoán ảnh
thuộc về lớp đó.

Danh sách các lớp mục tiêu được lưu trong thuộc tính `classes_`.

© Gemini Notebook

Đánh giá bộ phân loại đa lớp



Đánh giá

Ta vẫn có thể sử dụng hàm `cross_val_score` để đánh giá độ chính xác như phân loại nhị phân.



Chuẩn bị Dữ liệu

Để cải thiện kết quả, cần kết hợp với các bước Chuẩn bị Dữ liệu ở Chương 2.

Ví dụ: Chuẩn hóa tỉ lệ các đặc trưng (Feature Scaling) bằng `StandardScaler`.

© Gemini Notebook

3. Phân tích Lỗi & Các bài toán nâng cao

Trực quan hóa Ma trận nhầm lẫn đa lớp



1.

Đối với bài toán phân loại đa lớp (10 lớp), **ma trận nhầm lẫn** sẽ có kích thước 10×10 . Rất khó để nhìn bằng mắt thường.

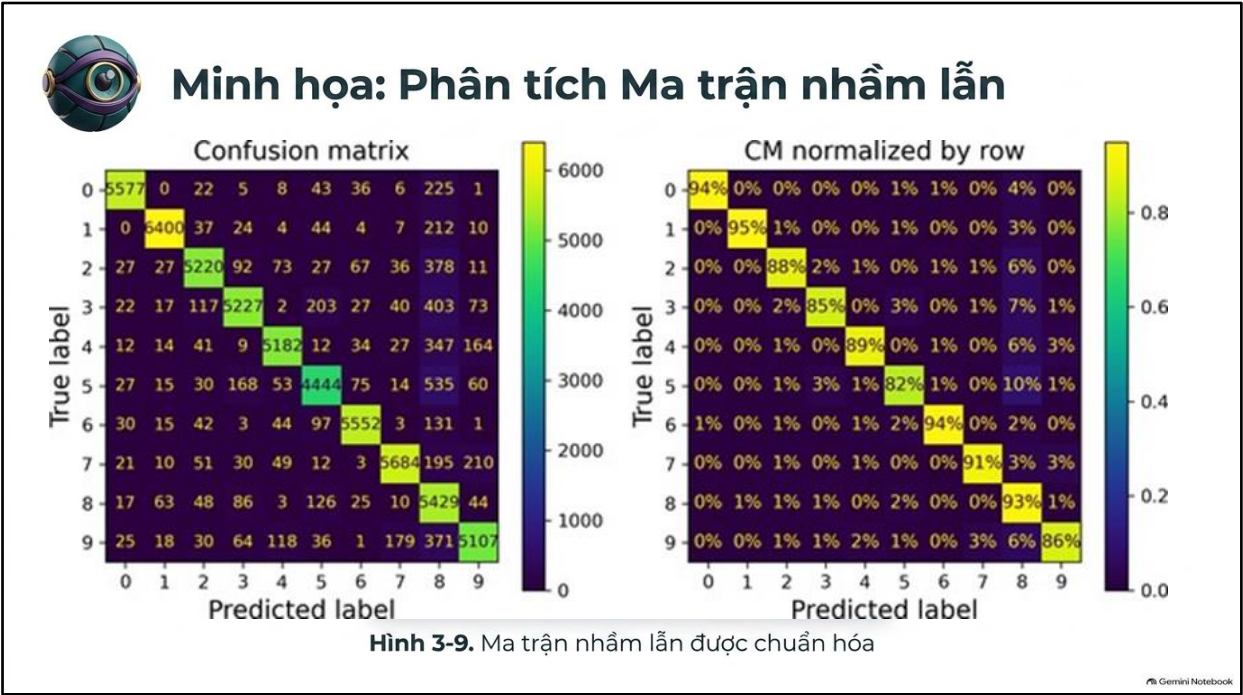
2.

Cách tốt nhất là trực quan hóa nó bằng **biểu đồ màu (heat map)**. Ở biểu đồ chuẩn, phần lớn dữ liệu hội tụ trên đường chéo (nghĩa là dự đoán đúng).

3.

Để xem phần **LỖI** rõ hơn, ta **tập trung trọng số** vào các ô 'dự đoán sai'.

© Gemini Notebook



Hình 3-10 (Ma trận nhầm lẫn chỉ hiển thị lỗi) trong tài liệu của bạn là một bước cải tiến cực kỳ thông minh để giải quyết nhược điểm của ma trận nhầm lẫn thông thường (Hình 3-9) [cite: 94, 150]. Ở ma trận nhầm lẫn thông thường, do mô hình phân loại đúng phần lớn dữ liệu nên **đường chéo chính sáng rực rỡ**, vô tình **che lấp đi các ô lỗi** (vốn có số lượng nhỏ hơn nhiều và hiển thị rất tối) [cite: 92, 151, 162]. Để **nhấn mạnh các lỗi (errors)** giúp ta dễ dàng phân tích và tìm hướng cải thiện mô hình, tác giả đã áp dụng đồng thời **hai kỹ thuật cốt lõi** dưới đây [cite: 93, 162]:

Kỹ thuật 1: Đặt trọng số bằng 0 cho các dự đoán đúng (Loại bỏ đường chéo chính)
Để loại bỏ sự xao nhãng từ các mẫu dự đoán đúng, chúng ta chỉ giữ lại và tập trung vào các mẫu dự đoán sai [cite: 93, 162]. Điều này được thực hiện bằng cách tạo ra một mảng trọng số trong Python [cite: 93, 174]:
sample_weight = (y_train_pred != y_train)
Cơ chế hoạt động: Phép so sánh này sẽ trả về giá trị True (tương đương với 1) cho các trường hợp dự đoán sai, và False (tương đương với 0) cho các trường hợp dự đoán đúng [cite: 93, 174].
Kết quả trực quan: Khi truyền sample_weight vào hàm vẽ [cite: 93, 174], toàn bộ các ô dự đoán đúng trên đường chéo chính sẽ bị nhân với 0 và **tối đen hoàn toàn** [cite: 93, 153]. Nhờ đó, mắt chúng ta sẽ không bị phân tâm và chỉ tập trung vào những ô lỗi phát sáng [cite: 93, 153].

Kỹ thuật 2: Chuẩn hóa ma trận lỗi (Normalization)
Nếu chỉ nhìn vào số lượng lỗi thô tuyệt đối, ta sẽ bị đánh lừa vì cỡ mẫu của các chữ số trong tập dữ liệu không đồng đều nhau [cite: 92, 151]. Vì vậy, việc **chuẩn hóa tỷ lệ phần trăm** là bắt buộc để có cái nhìn khách quan [cite: 92, 151]. **Hình 3-10** cung cấp hai góc nhìn chuẩn hóa bổ trợ cho nhau [cite: 94, 152]:
1. Biểu đồ bên trái: Chuẩn hóa lỗi theo hàng thực tế (normalize="true")
Biểu đồ này chia số lỗi của mỗi ô cho tổng số mẫu của lớp thực tế (tức là tổng hàng) [cite: 92, 93, 151, 152].
Câu hỏi trả lời: "Trong số tất cả các lỗi mà mô hình mắc phải trên một chữ số thực tế, chúng chủ yếu bị phân loại nhầm thành chữ số nào?" [cite: 95, 175]

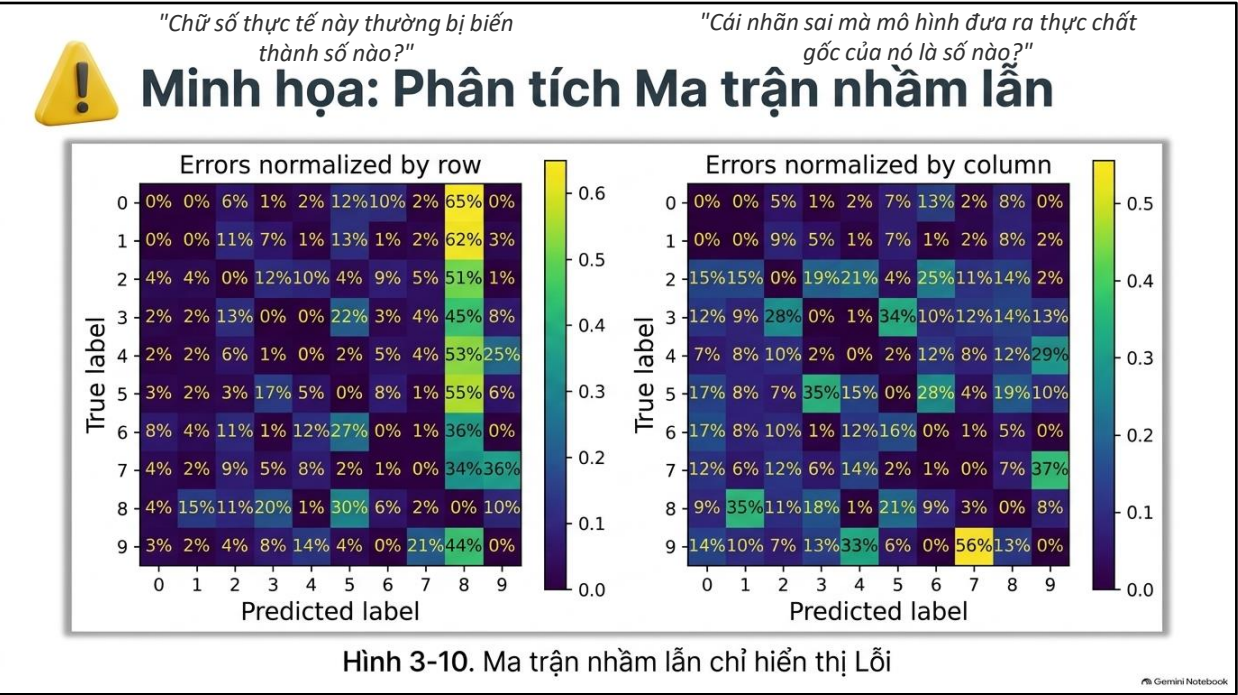
Cách đọc lỗi: Nhìn vào hàng thực tế của **số 7**, ta thấy ô giao với cột dự đoán **số 9** sáng lên với giá trị **36%** [cite: 95, 175]. Điều này có nghĩa là: **36% số lỗi mà mô hình mắc phải khi nhìn vào số 7 thực tế là phân loại nhầm nó thành số 9** [cite: 95, 175]. *(Lưu ý: Nó không có nghĩa là 36% tất cả số 7 bị đoán sai thành 9, mà chỉ tính riêng trong số các hình ảnh số 7 bị mô hình đoán sai)* [cite: 95, 175].

2. Biểu đồ bên phải: Chuẩn hóa lỗi theo cột dự đoán (normalize="pred")

Biểu đồ này chia số lỗi của mỗi ô cho tổng số mẫu của lớp được dự đoán (tức là tổng cột) [cite: 96, 176].

Câu hỏi trả lời: *"Trong số tất cả các hình ảnh bị mô hình dự đoán sai thành chữ số này, thực tế chúng chủ yếu là chữ số nào khác?"* [cite: 96, 176]

Cách đọc lỗi: Nhìn vào cột dự đoán **số 9**, ta thấy ô giao với hàng thực tế của **số 7** sáng rực lên với giá trị **56%** [cite: 96, 176]. Điều này có nghĩa là: **56% số lượng chữ số bị mô hình phân loại nhầm thành số 9 thực chất lại là những chữ số 7 viết tay** [cite: 96, 176].



Để hiểu cách tính toán ra các con số phần trăm hiển thị trong **Hình 3-10** (như **36%** hay **56%**), chúng ta cần đi từ ma trận lỗi thô và áp dụng từng bước tính toán đại số tuyến tính đơn giản được Scikit-Learn thực hiện tự động phía sau hậu trường [cite: 111, 113, 247, 248].

Quá trình tính toán được chia làm 3 bước cốt lõi như sau:

Bước 1: Triệt tiêu các dự đoán đúng (Đặt trọng số bằng 0)

Trước khi tính toán bất kỳ tỷ lệ phần trăm nào, chúng ta phải loại bỏ toàn bộ các dự đoán đúng nằm trên đường chéo chính của ma trận nhầm lẫn nhị phân [cite: 110, 246]. Điều này được thực hiện bằng

cách áp dụng mảng trọng số [cite: 111, 247]: `sample_weight = (y_train_pred != y_train)` [cite: 111, 247]

Với các mẫu dự đoán đúng ($y_{\text{dự đoán}} = y_{\text{thực tế}}$): Trọng số bằng 0\$ [cite: 110, 246]. Giá trị của ô đó bị triệt tiêu về 0\$ (tương ứng với các ô tối đen hoàn toàn trên đường chéo chính của đồ thị) [cite: 110, 246].

Với các mẫu dự đoán sai ($y_{\text{dự đoán}} \neq y_{\text{thực tế}}$): Trọng số bằng 1\$ [cite: 110, 246]. Giá trị của ô được giữ nguyên [cite: 110, 246].

Sau bước này, ta thu được một ma trận chỉ chứa số lượng các ca phân loại sai (lỗi) [cite: 110, 246].

Bước 2: Cách tính phần trăm ở Biểu đồ bên trái (Normalized by row)

Biểu đồ này hiển thị **Lỗi được chuẩn hóa theo hàng** (normalize="true") [cite: 109, 111, 245, 247]. Công thức tính cho từng ô là **lấy giá trị lỗi của ô đó chia cho tổng của cả hàng chứa nó** [cite: 109, 245].

Ý nghĩa toán học:
$$\text{Tỷ lệ ô (Hàng thực tế } i, \text{ Cột dự đoán } j) = \frac{\text{Số lượng mẫu lớp thực tế } i \text{ bị đoán sai thành lớp } j}{\text{Tổng số mẫu lớp thực tế } i \text{ bị đoán sai}}$$

Ví dụ thực tế với con số 36% (ở hàng #7, cột #9) [cite: 112, 247]:

Tử số: Ta đếm riêng số lượng ảnh thực tế là **số 7** bị mô hình đoán nhầm thành **số 9**. Gọi số lượng này là $E_{7 \rightarrow 9}$.

Mẫu số: Ta tính tổng số lượng ảnh thực tế là **số 7** bị mô hình phân loại sai (tức là tổng tất cả các lỗi xảy ra trên hàng 7, bao gồm đoán nhầm thành số 0, 1, 2, 3, 4, 5, 6, 8, 9). Gọi tổng số lỗi này là $E_{7\text{}}$.

Kết quả: Phép chia $\frac{E_{7 \rightarrow 9}}{E_7}$ cho ra kết quả chính xác là 36% [cite: 112, 247].

Ý nghĩa thực tế: Trong số tất cả các bức ảnh số 7 thực tế bị mô hình phân loại sai, có 36% trường hợp là bị mô hình nhận nhầm thành số 9 [cite: 112, 247].

Bước 3: Cách tính phần trăm ở Biểu đồ bên phải (Normalized by column)

Biểu đồ này hiển thị **Lỗi được chuẩn hóa theo cột** (normalize="pred") [cite: 113, 248]. Công thức tính cho từng ô là **lấy giá trị lỗi của ô đó chia cho tổng của cả cột chứa nó** [cite: 113, 248].

Ý nghĩa toán học:
$$\frac{\text{Tỷ lệ ô (Hàng thực tế } i, \text{ Cột dự đoán } j)}{\text{Số lượng mẫu lớp thực tế } i \text{ bị đoán sai thành lớp } j} = \frac{\text{Số lượng mẫu lớp thực tế } i \text{ bị đoán sai thành lớp } j}{\text{Tổng số mẫu bị đoán sai thành lớp } j}$$

Ví dụ thực tế với con số 56% (ở hàng #7, cột #9) [cite: 113, 248]:

Từ số: Vẫn là số lượng ảnh thực tế là **số 7** bị mô hình đoán nhầm thành **số 9** ($E_{7 \rightarrow 9}$).

Mẫu số: Ta tính tổng số lượng ảnh bị mô hình **phân loại sai thành số 9** (tức là tổng tất cả các lỗi xảy ra trên cột 9, bao gồm các ảnh thực tế là số 0, 1, 2, 3, 4, 5, 6, 7, 8 nhưng mô hình lại đoán nhầm thành số 9). Gọi tổng số lỗi này là $E_{\text{đoán nhầm thành 9}}$.

Kết quả: Phép chia $\frac{E_{7 \rightarrow 9}}{E_{\text{đoán nhầm thành 9}}}$ cho ra kết quả chính xác là 56% [cite: 113, 248].

Ý nghĩa thực tế: Trong số tất cả các chữ số viết tay bị mô hình đoán sai thành số 9, có đến 56% thực chất lại là những chữ số 7 viết tay [cite: 113, 248].

Tóm tắt nhanh cách đọc ma trận lỗi:

Khi đọc theo **Hàng ngang** (biểu đồ trái): Trả lời câu hỏi "*Chữ số thực tế này thường bị biến thành số nào?*" [cite: 110, 112, 246, 247]

Khi đọc theo **Cột dọc** (biểu đồ phải): Trả lời câu hỏi "*Cái nhãn sai mà mô hình đưa ra thực chất gốc của nó là số nào?*" [cite: 113, 248]



Phân tích lỗi riêng lẻ và cách khắc phục

Problem → Solution

Cách khắc phục

Vấn đề

Nhìn vào biểu đồ lỗi, ta có thể thấy số 8 thường bị nhầm lẫn nhiều nhất.



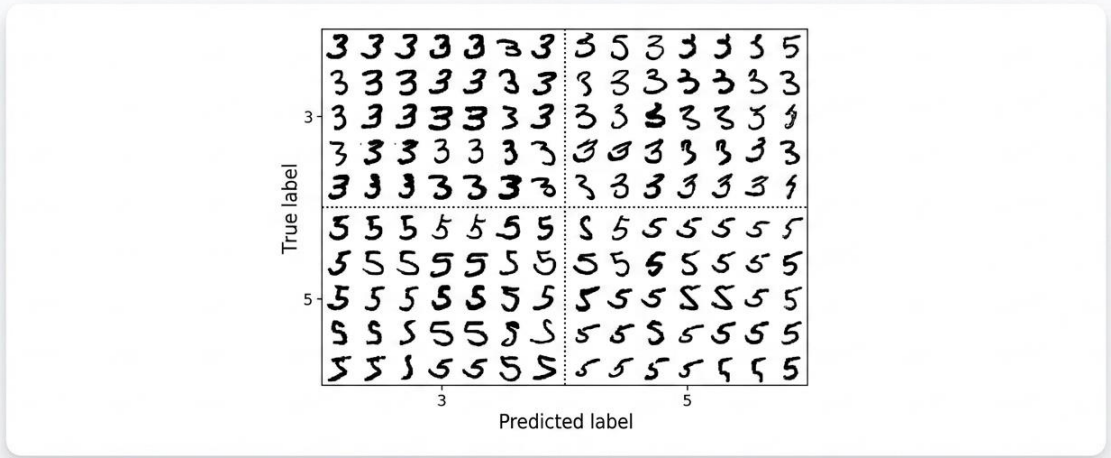
Thu thập thêm dữ liệu số 8 để mô hình học rõ hơn.

Viết thuật toán tạo đặc trưng (ví dụ: đếm số vòng kín của số 8).

Tiền xử lý hình ảnh, tăng độ tương phản...

Gemini Notebook

So sánh lỗi giữa số 3 và 5



Hình 3-11. Sự nhầm lẫn giữa chữ số 3 và 5

© Gemini Notebook

Tăng cường dữ liệu (Data Augmentation)



Mô hình máy học rất nhạy cảm với việc hình ảnh bị lệch (shift) hoặc xoay (rotate).
Giải pháp hiệu quả: Tăng cường dữ liệu (Data Augmentation).



Lấy các bức ảnh
huấn luyện gốc.



Biến đổi nhẹ (dịch trái,
dịch phải 1 pixel...)



Thêm vào tập huấn luyện, giúp
mô hình có khả năng chịu
đựng sai số (robust) tốt hơn!

© Gemini Notebook

Phân loại Đa nhãn (Multilabel Classification)



Phân loại Đa nhãn (Multilabel) là gì?

Một mẫu (instance) duy nhất nhưng thuộc về nhiều lớp cùng lúc.



Input

Ví dụ: Nhận diện khuôn mặt. Một bức ảnh có chứa cả Alice và Charlie, nhưng không có Bob.



Output

Đầu ra: Sẽ là một chuỗi cờ Nhị phân (Binary Tags).

[True]

[False]

[True]

© Gemini Notebook

Phân loại Đa nhãn (Multilabel Classification)



Phân loại Đa nhãn (Multilabel) là gì?

Một mẫu (instance) duy nhất nhưng thuộc về nhiều lớp cùng lúc.



Input

Ví dụ: Nhận diện khuôn mặt. Một bức ảnh có chứa cả Alice và Charlie, nhưng không có Bob.



Output

Đầu ra: Sẽ là một chuỗi cờ Nhị phân (Binary Tags).

[True]

[False]

[True]

© Gemini Notebook

Ví dụ về hệ thống đa nhãn



Áp dụng vào MNIST,
đánh giá 2 tiêu chí
cùng lúc:

Chữ số đó có lớn
không? (7, 8, 9)
→ `y_large`

Chữ số đó có phải là số
lẻ không? (1, 3, 5, 7, 9)
→ `y_odd`

Kết hợp 2 nhãn này
lại: `y_multilabel`.

Mô hình `KNeighborsClassifier` hỗ trợ xử lý việc này một cách tự nhiên.

© Gemini Notebook

Đánh giá hệ thống phân loại đa nhãn



Tính `F1-score` cho từng nhãn (từng tiêu chí) riêng biệt, sau đó lấy trung bình của tất cả các nhãn.

Tùy chọn `average="macro"`



Giả định mọi nhãn quan trọng như nhau.

Tùy chọn `average="weighted"`



Sẽ đánh trọng số dựa trên số lượng mẫu thực tế của từng nhãn (nếu tập dữ liệu bị mất cân bằng).

Phân loại đa nhãn với ClassifierChain

Nếu dùng mô hình như SVC (không hỗ trợ multilabel tự nhiên), ta có thể huấn luyện 1 mô hình cho mỗi nhãn. Tuy nhiên, các nhãn có thể phụ thuộc nhau (Ví dụ: số lớn > 7 thì có tỷ lệ số chẵn cao hơn).

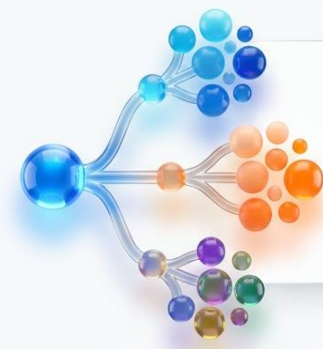


Lớp `ClassifierChain` giải quyết bằng cách tổ chức mô hình thành một chuỗi (chain).



© Gemini Notebook

Phân loại Đa đầu ra (Multioutput Classification)



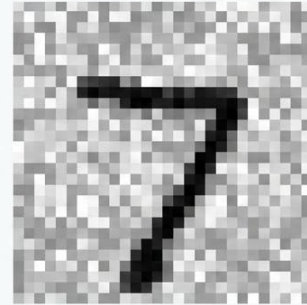
Phân loại Đa đầu ra là gì?

Khái quát hóa của Phân loại đa nhãn.

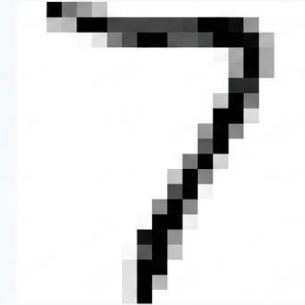
Đầu ra chứa nhiều nhãn.

Mỗi nhãn có thể là đa lớp (nhận nhiều giá trị khác nhau).

Minh họa: Quá trình khử nhiễu



Hình 3-12. Ảnh bị nhiễu
và Ảnh mục tiêu



Hình 3-13. Ảnh đã được
mô hình làm sạch

Bài toán khử nhiễu hình ảnh: Phương pháp



Chuẩn bị dữ liệu

Ta cố tình thêm nhiễu (noise) ngẫu nhiên vào dữ liệu huấn luyện.



Xác định mục tiêu

Mục tiêu mong muốn chính là hình ảnh MNIST ban đầu chưa nhiễu.

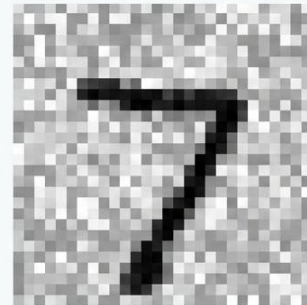


Huấn luyện mô hình

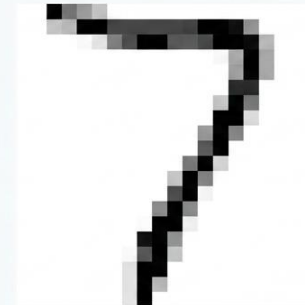
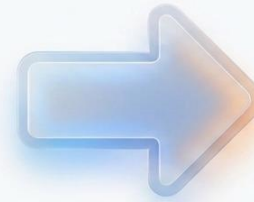
Mô hình KNeighborsClassifier sẽ học cách so khớp bức ảnh nhiễu và tìm lại bức ảnh gần gốc nhất.

Xem minh họa quá trình thực hiện ở Slide tiếp theo.

Minh họa: Quá trình khử nhiễu



Hình 3-12. Ảnh bị nhiễu
và Ảnh mục tiêu



Hình 3-13. Ảnh đã được
mô hình làm sạch

Tổng kết Chương 3



Hiểu rõ các chỉ số đánh giá cho Bài toán Phân loại (Precision, Recall, F1, ROC).



Kiểm soát và điều chỉnh được sự đánh đổi Precision/Recall bằng Threshold.



Nắm được quy trình Phân tích lỗi (Error Analysis) qua Ma trận nhầm lẫn.



Áp dụng các kỹ thuật Phân loại Đa lớp (OvR/OvO), Đa nhãn và Đa đầu ra cho các bài toán phức tạp.